

200 ETL Practice Tasks & Solutions

Hands-on T-SQL exercises built directly on the **DataWarehouse** project's bronze → silver → gold medallion architecture — covering CRM and ERP source integration, data cleaning, historization, dimensional modeling, and analytics.

Bronze Layer

Silver Layer

Gold / Star Schema

Window Functions

Data Quality

SCD & Historization

200

Practice Tasks

9

Categories

100%

Solutions Included

Table of Contents

Every task references the real bronze/silver/gold objects defined in the DataWarehouse project (schemas: **bronze**, **silver**, **gold**). Work through the categories in order for a full pipeline walkthrough, or jump to any section for targeted practice. Each task shows a difficulty badge (Beginner / Intermediate / Advanced) and a worked T-SQL solution.

A. Bronze Layer — Ingestion & Exploration

15 tasks

B. Data Quality & Profiling

30 tasks

C. Silver Layer Cleaning & Transformation

35 tasks

D. Historization & Slowly Changing Data

15 tasks

E. Cross-Source Integration (CRM + ERP)

20 tasks

F. Gold Layer — Dimensional Modeling

25 tasks

G. Gold Layer — Fact Table & Referential Integrity

20 tasks

H. Analytics & Aggregation on the Gold Layer

25 tasks

I. ETL Orchestration & Auditing

15 tasks

How to use this set

1. Restore or recreate the DataWarehouse schema from the provided DataWarehouse.sql script.
2. Attempt each task yourself before checking the solution — solutions show one valid approach, not the only one.
3. Run the validation-style tasks (Category B, C, F, G) after your own transformation code to self-check correctness.

A. Bronze Layer — Ingestion & Exploration

Tasks 1–15 • 15 practice tasks

#001 Bulk-load a bronze table

BEGINNER

Write a BULK INSERT statement to load bronze.crm_cust_info from 'cust_info.csv', skipping the header row and using a comma delimiter.

SOLUTION

```
BULK INSERT bronze.crm_cust_info
FROM 'D:\source_crm\cust_info.csv'
WITH (
    FIRSTROW = 2,
    FIELDTERMINATOR = ',',
    TABLOCK
);
```

- FIRSTROW = 2 skips the CSV header; TABLOCK improves bulk-load throughput.

#002 Row counts across all bronze tables

BEGINNER

Return the row count of every bronze table in a single result set.

SOLUTION

```
SELECT 'crm_cust_info' AS table_name, COUNT(*) AS row_count FROM bronze.crm_cust_info
UNION ALL SELECT 'crm_prd_info', COUNT(*) FROM bronze.crm_prd_info
UNION ALL SELECT 'crm_sales_details', COUNT(*) FROM bronze.crm_sales_details
UNION ALL SELECT 'erp_cust_az12', COUNT(*) FROM bronze.erp_cust_az12
UNION ALL SELECT 'erp_loc_a101', COUNT(*) FROM bronze.erp_loc_a101
UNION ALL SELECT 'erp_px_cat_glv2', COUNT(*) FROM bronze.erp_px_cat_glv2;
```

- A quick UNION ALL sanity check that every source landed before profiling begins.

#003 Preview raw sales data

BEGINNER

Preview the first 10 raw rows of bronze.crm_sales_details exactly as loaded.

SOLUTION

```
SELECT TOP (10) * FROM bronze.crm_sales_details;
```

- Always eyeball raw data before writing cleaning logic against it.

#004 Right-size a text column

INTERMEDIATE

Find the maximum character length actually used in `bronze.crm_cust_info.cst_firstname` to check whether `nvarchar(50)` is oversized or undersized.

SOLUTION

```
SELECT MAX(LEN(cst_firstname)) AS max_len, MIN(LEN(cst_firstname)) AS min_len FROM bronze.crm_cust_info;
```

- Column sizing should be driven by observed data, not guesswork.

#005 Full-row duplicates in bronze

INTERMEDIATE

Detect fully duplicated rows (every column identical) in `bronze.crm_prd_info`.

SOLUTION

```
SELECT prd_id, prd_key, prd_nm, prd_cost, prd_line, prd_start_dt, prd_end_dt, COUNT(*) AS cnt
FROM bronze.crm_prd_info
GROUP BY prd_id, prd_key, prd_nm, prd_cost, prd_line, prd_start_dt, prd_end_dt
HAVING COUNT(*) > 1;
```

- Full-row duplicates usually indicate a re-run load without truncation.

#006 Distinct marital-status codes

BEGINNER

List the distinct raw values of `cst_marital_status` in `bronze.crm_cust_info` to plan a normalization rule.

SOLUTION

```
SELECT DISTINCT cst_marital_status FROM bronze.crm_cust_info;
```

- Profiling distinct values first prevents surprises when writing CASE logic later.

#007 Distinct gender codes (ERP)

BEGINNER

List the distinct raw values of `gen` in `bronze.erp_cust_az12`.

SOLUTION

```
SELECT DISTINCT gen FROM bronze.erp_cust_az12;
```

- ERP gender codes are messier than CRM's — expect blanks, mixed case, and full words.

#008 Malformed integer dates

INTERMEDIATE

Find rows in `bronze.crm_sales_details` where `sls_order_dt` is not a valid 8-digit YYYYMMDD integer.

SOLUTION

```
SELECT sls_ord_num, sls_order_dt FROM bronze.crm_sales_details WHERE sls_order_dt <= 0 OR LEN(sls_order_dt) != 8;
```

- Dates arrive as raw integers in this source system, so length/format checks come before any CAST.

#009 Bronze load reconciliation

ADVANCED

Given a staging row count captured before truncation, write a query pattern that compares pre-load and post-load row counts per table to catch a failed or partial load.

SOLUTION

```
-- Capture counts before TRUNCATE, e.g. into #pre_counts(table_name, row_count)
-- Then after BULK INSERT, compare:
SELECT p.table_name, p.row_count AS before_load, c.row_count AS after_load
FROM #pre_counts p
JOIN (
    SELECT 'crm_cust_info' AS table_name, COUNT(*) AS row_count FROM bronze.crm_cust_info
) c ON p.table_name = c.table_name
WHERE p.row_count <> c.row_count;
```

- A reconciliation step catches silent truncation-without-reload failures.

#010 Whitespace scan on names

BEGINNER

Identify rows in `bronze.crm_cust_info` where `cst_firstname` or `cst_lastname` contain leading/trailing spaces.

SOLUTION

```
SELECT cst_id, cst_firstname, cst_lastname
FROM bronze.crm_cust_info
WHERE cst_firstname <> TRIM(cst_firstname) OR cst_lastname <> TRIM(cst_lastname);
```

- Untrimmed text breaks joins and skews DISTINCT/GROUP BY results downstream.

#011 Non-positive product cost

INTERMEDIATE

Find bronze.crm_prd_info rows with a NULL, zero, or negative prd_cost.

SOLUTION

```
SELECT prd_id, prd_key, prd_cost FROM bronze.crm_prd_info WHERE prd_cost IS NULL OR prd_cost <= 0;
```

- These rows will need an ISNULL/COALESCE default in the silver load.

#012 Sales formula violation in bronze

INTERMEDIATE

Find bronze.crm_sales_details rows where sls_sales does not equal sls_quantity * sls_price.

SOLUTION

```
SELECT sls_ord_num, sls_sales, sls_quantity, sls_price
FROM bronze.crm_sales_details
WHERE sls_sales <> sls_quantity * sls_price
OR sls_sales IS NULL OR sls_quantity IS NULL OR sls_price IS NULL;
```

- This is the exact check that motivates the sls_sales recalculation rule in the silver layer.

#013 Inconsistent location key formats

BEGINNER

Inspect bronze.erp_loc_a101.cid for inconsistent formatting (e.g. embedded dashes) that will block joins to crm_cust_info.cst_key.

SOLUTION

```
SELECT DISTINCT cid FROM bronze.erp_loc_a101 WHERE cid LIKE '%-%';
```

- A dash in one source's key and not the other is a classic ETL join-key mismatch.

#014 Safe reload with TRY/CATCH

ADVANCED

Write a script that truncates and reloads `bronze.erp_px_cat_g1v2`, wrapped in TRY/CATCH so a failed BULK INSERT is reported without crashing the batch.

SOLUTION

```
BEGIN TRY
    TRUNCATE TABLE bronze.erp_px_cat_g1v2;
    BULK INSERT bronze.erp_px_cat_g1v2
    FROM 'D:\source_erp\PX_CAT_G1V2.csv'
    WITH (FIRSTROW = 2, FIELDTERMINATOR = ',', TABLOCK);
END TRY
BEGIN CATCH
    PRINT 'Bronze load failed: ' + ERROR_MESSAGE();
END CATCH;
```

- TRY/CATCH keeps one bad file from silently corrupting the rest of the batch's logging.

#015 Bronze vs. silver column drift

INTERMEDIATE

Compare the column lists of `bronze.crm_prd_info` and `silver.crm_prd_info` to confirm which columns are added or split during transformation.

SOLUTION

```
SELECT COLUMN_NAME, TABLE_SCHEMA FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_NAME = 'crm_prd_info'
ORDER BY TABLE_SCHEMA, ORDINAL_POSITION;
```

- `silver.crm_prd_info` adds `cat_id` (split out of `prd_key`) and `dwh_create_date` versus `bronze`.

B. Data Quality & Profiling

Tasks 16–45 • 30 practice tasks

#016 NULL scan: bronze.crm_cust_info.cst_id

BEGINNER

Count how many rows in bronze.crm_cust_info have a NULL cst_id.

SOLUTION

```
SELECT COUNT(*) AS null_count FROM bronze.crm_cust_info WHERE cst_id IS NULL;
```

- A NULL business key almost always needs to be filtered out before deduplication.

#017 Duplicate key check: bronze.crm_cust_info.cst_id

INTERMEDIATE

Find values of cst_id in bronze.crm_cust_info that appear more than once.

SOLUTION

```
SELECT cst_id, COUNT(*) AS cnt FROM bronze.crm_cust_info GROUP BY cst_id HAVING COUNT(*) > 1;
```

- Duplicate keys must be resolved (e.g. keep-latest) before the column can anchor a dimension.

#018 Whitespace check: bronze.crm_cust_info.cst_firstname

BEGINNER

Find rows in bronze.crm_cust_info where cst_firstname has leading or trailing whitespace.

SOLUTION

```
SELECT cst_firstname FROM bronze.crm_cust_info WHERE cst_firstname <> TRIM(cst_firstname);
```

- Untrimmed values will fail exact-match joins even when they look identical.

#019 Whitespace check: bronze.crm_cust_info.cst_lastname

BEGINNER

Find rows in bronze.crm_cust_info where cst_lastname has leading or trailing whitespace.

SOLUTION

```
SELECT cst_lastname FROM bronze.crm_cust_info WHERE cst_lastname <> TRIM(cst_lastname);
```

- Untrimmed values will fail exact-match joins even when they look identical.

#020 Distinct values: bronze.crm_cust_info.cst_marital_status

BEGINNER

List the distinct values of cst_marital_status in bronze.crm_cust_info to design a normalization mapping.

SOLUTION

```
SELECT DISTINCT cst_marital_status FROM bronze.crm_cust_info ORDER BY cst_marital_status;
```

- Always enumerate real values before writing a CASE/normalization rule against them.

#021 Distinct values: bronze.crm_cust_info.cst_gndr

BEGINNER

List the distinct values of cst_gndr in bronze.crm_cust_info to design a normalization mapping.

SOLUTION

```
SELECT DISTINCT cst_gndr FROM bronze.crm_cust_info ORDER BY cst_gndr;
```

- Always enumerate real values before writing a CASE/normalization rule against them.

#022 Future-date check: bronze.crm_cust_info.cst_create_date

INTERMEDIATE

Find rows in bronze.crm_cust_info where cst_create_date is later than today's date.

SOLUTION

```
SELECT * FROM bronze.crm_cust_info WHERE cst_create_date > GETDATE();
```

- Future dates in a historical field are a strong sign of source-system data entry errors.

#023 NULL scan: bronze.crm_prd_info.prd_id

BEGINNER

Count how many rows in bronze.crm_prd_info have a NULL prd_id.

SOLUTION

```
SELECT COUNT(*) AS null_count FROM bronze.crm_prd_info WHERE prd_id IS NULL;
```

- A NULL business key almost always needs to be filtered out before deduplication.

#024 Duplicate key check: bronze.crm_prd_info.prd_key

INTERMEDIATE

Find values of prd_key in bronze.crm_prd_info that appear more than once.

SOLUTION

```
SELECT prd_key, COUNT(*) AS cnt FROM bronze.crm_prd_info GROUP BY prd_key HAVING COUNT(*) > 1;
```

- Duplicate keys must be resolved (e.g. keep-latest) before the column can anchor a dimension.

#025 Negative-value check: bronze.crm_prd_info.prd_cost

BEGINNER

Find rows in bronze.crm_prd_info where prd_cost is negative.

SOLUTION

```
SELECT * FROM bronze.crm_prd_info WHERE prd_cost < 0;
```

- Cost/price fields should never be negative; flag for correction or defaulting.

#026 Distinct values: bronze.crm_prd_info.prd_line

BEGINNER

List the distinct values of prd_line in bronze.crm_prd_info to design a normalization mapping.

SOLUTION

```
SELECT DISTINCT prd_line FROM bronze.crm_prd_info ORDER BY prd_line;
```

- Always enumerate real values before writing a CASE/normalization rule against them.

#027 Chronology check: prd_start_dt vs prd_end_dt

ADVANCED

Find rows in bronze.crm_prd_info where prd_start_dt is later than prd_end_dt, which should be impossible.

SOLUTION

```
SELECT * FROM bronze.crm_prd_info WHERE prd_start_dt > prd_end_dt;
```

- A start date after its own end date signals a source data entry defect.

#028 Whitespace check: bronze.crm_sales_details.sls_ord_num

BEGINNER

Find rows in bronze.crm_sales_details where sls_ord_num has leading or trailing whitespace.

SOLUTION

```
SELECT sls_ord_num FROM bronze.crm_sales_details WHERE sls_ord_num <> TRIM(sls_ord_num);
```

- Untrimmed values will fail exact-match joins even when they look identical.

#029 Invalid integer-date check: bronze.crm_sales_details.sls_order_dt

INTERMEDIATE

sls_order_dt is stored as an integer YYYYMMDD. Find rows where it is 0 or not exactly 8 digits long.

SOLUTION

```
SELECT * FROM bronze.crm_sales_details WHERE sls_order_dt <= 0 OR LEN(sls_order_dt) <> 8;
```

- These rows must be converted to NULL, not cast directly, or the CAST will error out.

#030 Invalid integer-date check: bronze.crm_sales_details.sls_ship_dt

INTERMEDIATE

sls_ship_dt is stored as an integer YYYYMMDD. Find rows where it is 0 or not exactly 8 digits long.

SOLUTION

```
SELECT * FROM bronze.crm_sales_details WHERE sls_ship_dt <= 0 OR LEN(sls_ship_dt) <> 8;
```

- These rows must be converted to NULL, not cast directly, or the CAST will error out.

#031 Invalid integer-date check: bronze.crm_sales_details.sls_due_dt

INTERMEDIATE

sls_due_dt is stored as an integer YYYYMMDD. Find rows where it is 0 or not exactly 8 digits long.

SOLUTION

```
SELECT * FROM bronze.crm_sales_details WHERE sls_due_dt <= 0 OR LEN(sls_due_dt) <> 8;
```

- These rows must be converted to NULL, not cast directly, or the CAST will error out.

#032 Order-vs-ship chronology: sls_order_dt vs sls_ship_dt

ADVANCED

Find rows in bronze.crm_sales_details where sls_order_dt (order date) is after sls_ship_dt (ship date).

SOLUTION

```
SELECT * FROM bronze.crm_sales_details WHERE sls_order_dt > sls_ship_dt;
```

- Shipping before ordering is a logical impossibility worth flagging to the source team.

#033 Non-positive value check: bronze.crm_sales_details.sls_sales

BEGINNER

Find rows in bronze.crm_sales_details where sls_sales is NULL, zero, or negative.

SOLUTION

```
SELECT * FROM bronze.crm_sales_details WHERE sls_sales IS NULL OR sls_sales <= 0;
```

- Non-positive sales/price values need a recalculation rule rather than a silent pass-through.

#034 Non-positive value check: bronze.crm_sales_details.sls_price

BEGINNER

Find rows in bronze.crm_sales_details where sls_price is NULL, zero, or negative.

SOLUTION

```
SELECT * FROM bronze.crm_sales_details WHERE sls_price IS NULL OR sls_price <= 0;
```

- Non-positive sales/price values need a recalculation rule rather than a silent pass-through.

#035 Cross-column formula check: `sls_sales = sls_quantity * sls_price`

ADVANCED

Find rows in `bronze.crm_sales_details` where `sls_sales` does not equal `sls_quantity * sls_price`.

SOLUTION

```
SELECT * FROM bronze.crm_sales_details WHERE sls_sales <> sls_quantity * ABS(sls_price) OR sls_sales IS NULL;
```

- This check is the basis for the `sls_sales` recalculation rule used in the silver load.

#036 Prefix format check: `bronze.erp_cust_az12.cid`

INTERMEDIATE

Find rows in `bronze.erp_cust_az12` where `cid` begins with the 'NAS' prefix that needs stripping before it can join to CRM keys.

SOLUTION

```
SELECT cid FROM bronze.erp_cust_az12 WHERE cid LIKE 'NAS%';
```

- ERP customer IDs sometimes carry a system prefix that CRM keys never have.

#037 Future-date check: `bronze.erp_cust_az12.bdate`

INTERMEDIATE

Find rows in `bronze.erp_cust_az12` where `bdate` is later than today's date.

SOLUTION

```
SELECT * FROM bronze.erp_cust_az12 WHERE bdate > GETDATE();
```

- Future dates in a historical field are a strong sign of source-system data entry errors.

#038 Implausible-age check: `bronze.erp_cust_az12.bdate`

INTERMEDIATE

Find rows in `bronze.erp_cust_az12` where `bdate` implies an age over 100 years, a likely data entry error.

SOLUTION

```
SELECT * FROM bronze.erp_cust_az12 WHERE bdate < DATEADD(YEAR, -100, GETDATE());
```

- Implausible ages are usually typos in the year (e.g. 1902 instead of 1992).

#039 Distinct values: bronze.erp_cust_az12.gen

BEGINNER

List the distinct values of gen in bronze.erp_cust_az12 to design a normalization mapping.

SOLUTION

```
SELECT DISTINCT gen FROM bronze.erp_cust_az12 ORDER BY gen;
```

- Always enumerate real values before writing a CASE/normalization rule against them.

#040 Dash format check: bronze.erp_loc_a101.cid

BEGINNER

Find rows in bronze.erp_loc_a101 where cid contains a dash that must be removed before joining to crm_cust_info.cst_key.

SOLUTION

```
SELECT DISTINCT cid FROM bronze.erp_loc_a101 WHERE cid LIKE '%-%';
```

- Removing punctuation from a join key is one of the most common silver-layer fixes.

#041 Distinct values: bronze.erp_loc_a101.cntry

BEGINNER

List the distinct values of cntry in bronze.erp_loc_a101 to design a normalization mapping.

SOLUTION

```
SELECT DISTINCT cntry FROM bronze.erp_loc_a101 ORDER BY cntry;
```

- Always enumerate real values before writing a CASE/normalization rule against them.

#042 Distinct values: bronze.erp_px_cat_g1v2.cat

BEGINNER

List the distinct values of cat in bronze.erp_px_cat_g1v2 to design a normalization mapping.

SOLUTION

```
SELECT DISTINCT cat FROM bronze.erp_px_cat_g1v2 ORDER BY cat;
```

- Always enumerate real values before writing a CASE/normalization rule against them.

#043 Distinct values: bronze.erp_px_cat_g1v2.subcat

BEGINNER

List the distinct values of subcat in bronze.erp_px_cat_g1v2 to design a normalization mapping.

SOLUTION

```
SELECT DISTINCT subcat FROM bronze.erp_px_cat_g1v2 ORDER BY subcat;
```

- Always enumerate real values before writing a CASE/normalization rule against them.

#044 Distinct values: bronze.erp_px_cat_g1v2.maintenance

BEGINNER

List the distinct values of maintenance in bronze.erp_px_cat_g1v2 to design a normalization mapping.

SOLUTION

```
SELECT DISTINCT maintenance FROM bronze.erp_px_cat_g1v2 ORDER BY maintenance;
```

- Always enumerate real values before writing a CASE/normalization rule against them.

#045 Key length consistency: bronze.crm_cust_info.cst_key

INTERMEDIATE

Check whether cst_key in bronze.crm_cust_info has a consistent character length, which helps validate the key format before joining.

SOLUTION

```
SELECT LEN(cst_key) AS key_len, COUNT(*) AS cnt FROM bronze.crm_cust_info GROUP BY LEN(cst_key);
```

- An inconsistent key length across rows often signals mixed formats from different source batches.

C. Silver Layer Cleaning & Transformation

Tasks 46-80 • 35 practice tasks

#046 Deduplicate customers, keep latest

INTERMEDIATE

From `bronze.crm_cust_info`, keep only the most recent record per `cst_id` (by `cst_create_date`) and discard NULL `cst_id` rows.

SOLUTION

```
SELECT *
FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY cst_id ORDER BY cst_create_date DESC) AS flag_last
  FROM bronze.crm_cust_info
  WHERE cst_id IS NOT NULL
) t
WHERE flag_last = 1;
```

- `ROW_NUMBER()` over a partition is the standard 'keep the latest' deduplication pattern.

#047 Trim customer name fields

BEGINNER

Write a `SELECT` that trims `cst_firstname` and `cst_lastname`.

SOLUTION

```
SELECT cst_id, TRIM(cst_firstname) AS cst_firstname, TRIM(cst_lastname) AS cst_lastname FROM bronze.crm_cust_info;
```

- `TRIM()` (SQL Server 2017+) replaces the older `LTRIM(RTRIM(...))` nesting.

#048 Normalize marital status

BEGINNER

Map cst_marital_status codes S/M to 'Single'/'Married', defaulting anything else to 'n/a'.

SOLUTION

```
SELECT cst_id,  
       CASE UPPER(TRIM(cst_marital_status))  
         WHEN 'S' THEN 'Single'  
         WHEN 'M' THEN 'Married'  
         ELSE 'n/a'  
       END AS cst_marital_status  
FROM bronze.crm_cust_info;
```

- UPPER(TRIM(...)) before the CASE guards against stray case and whitespace variants.

#049 Normalize CRM gender

BEGINNER

Map cst_gndr codes F/M to 'Female'/'Male', defaulting to 'n/a'.

SOLUTION

```
SELECT cst_id,  
       CASE UPPER(TRIM(cst_gndr))  
         WHEN 'F' THEN 'Female'  
         WHEN 'M' THEN 'Male'  
         ELSE 'n/a'  
       END AS cst_gndr  
FROM bronze.crm_cust_info;
```

- Consistent 'n/a' defaults (not NULL) simplify downstream reporting and COALESCE logic.

#050 Split a composite product key

INTERMEDIATE

prd_key stores a 5-character category code followed by an underscore/dash and the real product key (e.g. 'CO-RF-FR-R92B-58'). Extract the category id and the product key into two columns.

SOLUTION

```
SELECT  
  prd_id,  
  REPLACE(SUBSTRING(prd_key, 1, 5), '-', '_') AS cat_id,  
  SUBSTRING(prd_key, 7, LEN(prd_key)) AS prd_key  
FROM bronze.crm_prd_info;
```

- Splitting a composite natural key is common when the source system embeds a category code in the SKU.

#051 Default missing product cost

BEGINNER

Replace NULL prd_cost values with 0.

SOLUTION

```
SELECT prd_id, ISNULL(prd_cost, 0) AS prd_cost FROM bronze.crm_prd_info;
```

- ISNULL/COALESCE with a sane default avoids NULL-propagation errors in later cost aggregations.

#052 Decode product line codes

BEGINNER

Map prd_line codes M/R/S/T to Mountain/Road/'Other Sales'/Touring, defaulting to 'n/a'.

SOLUTION

```
SELECT prd_id,
       CASE UPPER(TRIM(prd_line))
         WHEN 'M' THEN 'Mountain'
         WHEN 'R' THEN 'Road'
         WHEN 'S' THEN 'Other Sales'
         WHEN 'T' THEN 'Touring'
         ELSE 'n/a'
       END AS prd_line
FROM bronze.crm_prd_info;
```

- Decoding single-letter codes into business-readable labels belongs in the silver layer, not the gold reports.

#053 Derive product end-date from the next version

ADVANCED

For each prd_key, derive prd_end_dt as one day before the next row's prd_start_dt (ordered by start date).

SOLUTION

```
SELECT prd_id, prd_key, prd_start_dt,
       CAST(LEAD(prd_start_dt) OVER (PARTITION BY prd_key ORDER BY prd_start_dt) - 1 AS DATE) AS prd_end_dt
FROM bronze.crm_prd_info;
```

- LEAD() looks at the next row in the same partition — a clean way to infer effective-end dates without a self-join.

#054 Safely cast integer order dates

INTERMEDIATE

Convert `sls_order_dt` from an 8-digit integer to a `DATE`, returning `NULL` for 0 or wrong-length values instead of erroring.

SOLUTION

```
SELECT sls_ord_num,  
       CASE WHEN sls_order_dt = 0 OR LEN(sls_order_dt) <> 8 THEN NULL  
            ELSE CAST(CAST(sls_order_dt AS VARCHAR) AS DATE)  
       END AS sls_order_dt  
FROM bronze.crm_sales_details;
```

- Guard the `CAST` with a validity check first — `CAST`ing an invalid integer directly to `DATE` throws a runtime error.

#055 Safely cast ship and due dates

INTERMEDIATE

Apply the same safe-cast pattern used for `sls_order_dt` to `sls_ship_dt` and `sls_due_dt`.

SOLUTION

```
SELECT sls_ord_num,  
       CASE WHEN sls_ship_dt = 0 OR LEN(sls_ship_dt) <> 8 THEN NULL ELSE CAST(CAST(sls_ship_dt AS VARCHAR) AS  
DATE) END AS sls_ship_dt,  
       CASE WHEN sls_due_dt = 0 OR LEN(sls_due_dt) <> 8 THEN NULL ELSE CAST(CAST(sls_due_dt AS VARCHAR) AS  
DATE) END AS sls_due_dt  
FROM bronze.crm_sales_details;
```

- Reusing the same validation pattern across similar columns keeps the transformation logic consistent and easy to review.

#056 Recalculate sales amount

ADVANCED

Recalculate `sls_sales` as `quantity * ABS(price)` whenever the stored value is missing, non-positive, or inconsistent with `quantity*price`.

SOLUTION

```
SELECT sls_ord_num,  
       CASE  
         WHEN sls_sales IS NULL OR sls_sales <= 0 OR sls_sales <> sls_quantity * ABS(sls_price)  
         THEN sls_quantity * ABS(sls_price)  
         ELSE sls_sales  
       END AS sls_sales  
FROM bronze.crm_sales_details;
```

- Recomputing from trusted quantity/price is safer than trying to 'fix' an inconsistent stored total.

#057 Derive missing price

ADVANCED

Derive `sls_price` as `sls_sales / sls_quantity` whenever the stored price is NULL or non-positive, guarding against division by zero.

SOLUTION

```
SELECT sls_ord_num,
       CASE
         WHEN sls_price IS NULL OR sls_price <= 0
           THEN sls_sales / NULLIF(sls_quantity, 0)
         ELSE sls_price
       END AS sls_price
FROM bronze.crm_sales_details;
```

- `NULLIF(sls_quantity, 0)` turns a would-be divide-by-zero error into a clean NULL result.

#058 Strip ERP customer-id prefix

BEGINNER

Remove the 'NAS' prefix from `erp_cust_az12.cid` where present.

SOLUTION

```
SELECT
  CASE WHEN cid LIKE 'NAS%' THEN SUBSTRING(cid, 4, LEN(cid)) ELSE cid END AS cid
FROM bronze.erp_cust_az12;
```

- Stripping a system-specific prefix is required before this key can join to `crm_cust_info.cst_key`.

#059 Nullify future birthdates

BEGINNER

Set `bdate` to NULL in `erp_cust_az12` whenever it is later than today.

SOLUTION

```
SELECT cid, CASE WHEN bdate > GETDATE() THEN NULL ELSE bdate END AS bdate FROM bronze.erp_cust_az12;
```

- An impossible birthdate is better represented as an honest NULL than left as bad data.

#060 Normalize ERP gender values

BEGINNER

Normalize `erp_cust_az12.gen`: F/FEMALE -> 'Female', M/MALE -> 'Male', anything else -> 'n/a'.

SOLUTION

```
SELECT cid,
       CASE
         WHEN UPPER(TRIM(gen)) IN ('F','FEMALE') THEN 'Female'
         WHEN UPPER(TRIM(gen)) IN ('M','MALE') THEN 'Male'
         ELSE 'n/a'
       END AS gen
FROM bronze.erp_cust_az12;
```

- ERP gender values need a broader IN-list than CRM's since the source stores both codes and full words.

#061 Strip dashes from location keys

BEGINNER

Remove dashes from `erp_loc_a101.cid` so it matches the format of `crm_cust_info.cst_key`.

SOLUTION

```
SELECT REPLACE(cid, '-', '') AS cid, cntry FROM bronze.erp_loc_a101;
```

- REPLACE is the simplest fix when two systems format the same logical key differently.

#062 Normalize country names

BEGINNER

Normalize `erp_loc_a101.cntry`: 'DE' -> 'Germany', 'US'/'USA' -> 'United States', blank/NULL -> 'n/a', else trim as-is.

SOLUTION

```
SELECT cid,
       CASE
         WHEN TRIM(cntry) = 'DE' THEN 'Germany'
         WHEN TRIM(cntry) IN ('US','USA') THEN 'United States'
         WHEN TRIM(cntry) = '' OR cntry IS NULL THEN 'n/a'
         ELSE TRIM(cntry)
       END AS cntry
FROM bronze.erp_loc_a101;
```

- Country-code normalization is a very common silver-layer task across nearly every ERP integration.

#063 Straight pass-through load

BEGINNER

Confirm `erp_px_cat_glv2` needs no transformation and write the direct silver load statement.

SOLUTION

```
INSERT INTO silver.erp_px_cat_glv2 (id, cat, subcat, maintenance) SELECT id, cat, subcat, maintenance FROM bronze.erp_px_cat_glv2;
```

- Not every bronze table needs cleaning — verify with profiling first rather than adding logic by default.

#064 Populate the `dwh_create_date` audit column

BEGINNER

Explain and demonstrate how `dwh_create_date` gets populated automatically on insert into a silver table.

SOLUTION

```
ALTER TABLE silver.crm_cust_info ADD DEFAULT (GETDATE()) FOR dwh_create_date;
```

- A `DEFAULT` constraint means the load `INSERT` never has to reference the audit column explicitly.

#065 Full silver load: `crm_cust_info`

INTERMEDIATE

Write the complete `TRUNCATE + INSERT` statement that loads `silver.crm_cust_info` from bronze, applying dedup, trim, and code-normalization rules together.

SOLUTION

```
TRUNCATE TABLE silver.crm_cust_info;
INSERT INTO silver.crm_cust_info (cst_id, cst_key, cst_firstname, cst_lastname, cst_marital_status, cst_gndr, cst_create_date)
SELECT cst_id, cst_key, TRIM(cst_firstname), TRIM(cst_lastname),
       CASE UPPER(TRIM(cst_marital_status)) WHEN 'S' THEN 'Single' WHEN 'M' THEN 'Married' ELSE 'n/a' END,
       CASE UPPER(TRIM(cst_gndr)) WHEN 'F' THEN 'Female' WHEN 'M' THEN 'Male' ELSE 'n/a' END,
       cst_create_date
FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY cst_id ORDER BY cst_create_date DESC) AS flag_last
  FROM bronze.crm_cust_info WHERE cst_id IS NOT NULL
) t WHERE flag_last = 1;
```

- This combines every `crm_cust_info` rule from this section into the single production load statement.

#066 Full silver load: crm_prd_info

INTERMEDIATE

Write the complete TRUNCATE + INSERT statement that loads silver.crm_prd_info, including key-splitting, cost defaulting, line decoding, and end-date derivation.

SOLUTION

```
TRUNCATE TABLE silver.crm_prd_info;
INSERT INTO silver.crm_prd_info (prd_id, cat_id, prd_key, prd_nm, prd_cost, prd_line, prd_start_dt, prd_end_dt)
SELECT prd_id,
       REPLACE(SUBSTRING(prd_key, 1, 5), '-', '_'),
       SUBSTRING(prd_key, 7, LEN(prd_key)),
       prd_nm,
       ISNULL(prd_cost, 0),
       CASE UPPER(TRIM(prd_line)) WHEN 'M' THEN 'Mountain' WHEN 'R' THEN 'Road' WHEN 'S' THEN 'Other Sales' WHEN '
T' THEN 'Touring' ELSE 'n/a' END,
       CAST(prd_start_dt AS DATE),
       CAST(LEAD(prd_start_dt) OVER (PARTITION BY prd_key ORDER BY prd_start_dt) - 1 AS DATE)
FROM bronze.crm_prd_info;
```

- Bringing all crm_prd_info rules together in one statement is exactly what the production stored procedure does.

#067 Full silver load: crm_sales_details

INTERMEDIATE

Write the complete TRUNCATE + INSERT statement for silver.crm_sales_details combining safe date casting, sales recalculation, and price derivation.

SOLUTION

```
TRUNCATE TABLE silver.crm_sales_details;
INSERT INTO silver.crm_sales_details (sls_ord_num, sls_prd_key, sls_cust_id, sls_order_dt, sls_ship_dt, sls_due_dt, sls_sales, sls_quantity, sls_price)
SELECT sls_ord_num, sls_prd_key, sls_cust_id,
       CASE WHEN sls_order_dt = 0 OR LEN(sls_order_dt) <> 8 THEN NULL ELSE CAST(CAST(sls_order_dt AS VARCHAR) AS DATE) END,
       CASE WHEN sls_ship_dt = 0 OR LEN(sls_ship_dt) <> 8 THEN NULL ELSE CAST(CAST(sls_ship_dt AS VARCHAR) AS DATE) END,
       CASE WHEN sls_due_dt = 0 OR LEN(sls_due_dt) <> 8 THEN NULL ELSE CAST(CAST(sls_due_dt AS VARCHAR) AS DATE) END,
       CASE WHEN sls_sales IS NULL OR sls_sales <= 0 OR sls_sales <> sls_quantity * ABS(sls_price)
            THEN sls_quantity * ABS(sls_price) ELSE sls_sales END,
       sls_quantity,
       CASE WHEN sls_price IS NULL OR sls_price <= 0 THEN sls_sales / NULLIF(sls_quantity, 0) ELSE sls_price END
FROM bronze.crm_sales_details;
```

- This is the most rule-dense table in the whole pipeline — dates, sales, and price all get corrected in one pass.

#068 Full silver load: erp_cust_az12

INTERMEDIATE

Write the complete TRUNCATE + INSERT statement for silver.erp_cust_az12 combining prefix stripping, future-date nulling, and gender normalization.

SOLUTION

```
TRUNCATE TABLE silver.erp_cust_az12;
INSERT INTO silver.erp_cust_az12 (cid, bdate, gen)
SELECT
  CASE WHEN cid LIKE 'NAS%' THEN SUBSTRING(cid, 4, LEN(cid)) ELSE cid END,
  CASE WHEN bdate > GETDATE() THEN NULL ELSE bdate END,
  CASE WHEN UPPER(TRIM(gen)) IN ('F','FEMALE') THEN 'Female' WHEN UPPER(TRIM(gen)) IN ('M','MALE') THEN 'Male' ELSE 'n/a' END
FROM bronze.erp_cust_az12;
```

- Every column in this table needed its own cleaning rule — a good example of how uneven data quality can be across a single source.

#069 Full silver load: erp_loc_a101

INTERMEDIATE

Write the complete TRUNCATE + INSERT statement for silver.erp_loc_a101 combining dash removal and country normalization.

SOLUTION

```
TRUNCATE TABLE silver.erp_loc_a101;
INSERT INTO silver.erp_loc_a101 (cid, cntry)
SELECT REPLACE(cid, '-', ''),
  CASE WHEN TRIM(cntry) = 'DE' THEN 'Germany' WHEN TRIM(cntry) IN ('US','USA') THEN 'United States'
  WHEN TRIM(cntry) = '' OR cntry IS NULL THEN 'n/a' ELSE TRIM(cntry) END
FROM bronze.erp_loc_a101;
```

- A small table can still require several distinct cleaning rules — don't assume simplicity from row/column count.

#070 Full silver load: erp_px_cat_g1v2 with logging

BEGINNER

Write the TRUNCATE + INSERT for silver.erp_px_cat_g1v2 with PRINT statements reporting progress, matching the pattern used elsewhere in the pipeline.

SOLUTION

```
PRINT '>> Truncating Table: silver.erp_px_cat_g1v2';
TRUNCATE TABLE silver.erp_px_cat_g1v2;
PRINT '>> Inserting Data Into: silver.erp_px_cat_g1v2';
INSERT INTO silver.erp_px_cat_g1v2 (id, cat, subcat, maintenance)
SELECT id, cat, subcat, maintenance FROM bronze.erp_px_cat_g1v2;
```

- Consistent PRINT logging across every table load makes batch output easy to scan for failures.

#071 Post-load duplicate check on silver

INTERMEDIATE

After loading silver.crm_cust_info, verify no duplicate cst_id values remain.

SOLUTION

```
SELECT cst_id, COUNT(*) FROM silver.crm_cust_info GROUP BY cst_id HAVING COUNT(*) > 1;
```

- A validation query after every load is what turns 'we cleaned the data' into 'we proved the data is clean'.

#072 Post-load date order check on silver

INTERMEDIATE

After loading silver.crm_prd_info, verify prd_start_dt is never later than prd_end_dt.

SOLUTION

```
SELECT * FROM silver.crm_prd_info WHERE prd_start_dt > prd_end_dt;
```

- This should return zero rows if the LEAD()-based end-date derivation was implemented correctly.

#073 Post-load NULL date audit on silver

BEGINNER

Count how many rows in silver.crm_sales_details still have a NULL sls_order_dt after the safe-cast transformation.

SOLUTION

```
SELECT COUNT(*) AS still_null FROM silver.crm_sales_details WHERE sls_order_dt IS NULL;
```

- A non-zero count here is expected if the source genuinely contained invalid dates — it documents data loss, not a bug.

#074 Bronze-to-silver row count parity

INTERMEDIATE

Compare row counts between bronze and silver for crm_cust_info to confirm the expected reduction from deduplication.

SOLUTION

```
SELECT  
  (SELECT COUNT(*) FROM bronze.crm_cust_info) AS bronze_rows,  
  (SELECT COUNT(*) FROM silver.crm_cust_info) AS silver_rows;
```

- silver_rows should equal the count of distinct non-null cst_id values in bronze — verify the arithmetic, don't just eyeball it.

#075 Catch whitespace-only strings

INTERMEDIATE

Extend the marital-status normalization to also treat an empty or whitespace-only string as 'n/a' (not just NULL).

SOLUTION

```
SELECT cst_id,  
       CASE WHEN cst_marital_status IS NULL OR TRIM(cst_marital_status) = '' THEN 'n/a'  
            WHEN UPPER(TRIM(cst_marital_status)) = 'S' THEN 'Single'  
            WHEN UPPER(TRIM(cst_marital_status)) = 'M' THEN 'Married'  
            ELSE 'n/a'  
       END AS cst_marital_status  
FROM bronze.crm_cust_info;
```

- An empty string is not NULL in SQL — IS NULL alone will silently miss blank-string rows.

#076 Reusable normalization pattern

ADVANCED

Generalize the UPPER(TRIM(...)) CASE pattern used across cst_marital_status, cst_gndr, prd_line, and gen into a written checklist you could apply to any new categorical column.

SOLUTION

```
-- Pattern:  
-- 1) TRIM to remove whitespace  
-- 2) UPPER to remove case sensitivity  
-- 3) CASE/IN-list mapping known codes to business labels  
-- 4) ELSE 'n/a' catch-all so unmapped codes never silently pass through as NULL
```

- Writing the pattern down once turns four separate fixes into one reusable rule for any future categorical column.

#077 Idempotent silver load

INTERMEDIATE

Explain, with a query pattern, why TRUNCATE-then-INSERT makes the silver load safe to re-run multiple times without creating duplicates.

SOLUTION

```
TRUNCATE TABLE silver.crm_cust_info;  
-- followed by a full INSERT ... SELECT from bronze  
-- re-running this script always yields the same end state
```

- TRUNCATE-then-full-reload is the simplest way to guarantee idempotency for a full-refresh (non-incremental) pipeline.

#078 Post-transform value-domain check

ADVANCED

Verify that after transformation, silver.crm_cust_info.cst_gndr contains only the three expected values: Female, Male, n/a.

SOLUTION

```
SELECT DISTINCT cst_gndr FROM silver.crm_cust_info WHERE cst_gndr NOT IN ('Female','Male','n/a');
```

- An empty result set here is a passing test — any row returned means an unmapped code slipped through.

#079 Cross-source key-format check

INTERMEDIATE

After cleaning, verify that silver.erp_cust_az12.cid values now match the character length/format of silver.crm_cust_info.cst_key.

SOLUTION

```
SELECT DISTINCT LEN(cid) AS erp_key_len FROM silver.erp_cust_az12  
EXCEPT  
SELECT DISTINCT LEN(cst_key) AS len FROM silver.crm_cust_info;
```

- An empty result confirms the two cleaned key formats are now join-compatible.

#080 Consolidated silver data-quality report

ADVANCED

Combine several silver validation checks (duplicate cst_id, bad date order, unmapped gender) into a single report query, one labeled row per check.

SOLUTION

```
SELECT 'Duplicate cst_id' AS check_name, COUNT(*) AS failing_rows  
FROM (SELECT cst_id FROM silver.crm_cust_info GROUP BY cst_id HAVING COUNT(*) > 1) x  
UNION ALL  
SELECT 'prd_start_dt > prd_end_dt', COUNT(*) FROM silver.crm_prd_info WHERE prd_start_dt > prd_end_dt  
UNION ALL  
SELECT 'Unmapped cst_gndr', COUNT(*) FROM silver.crm_cust_info WHERE cst_gndr NOT IN ('Female','Male','n/a');
```

- A single-query DQ scorecard is easy to run after every load and easy to alert on when any failing_rows > 0.

D. Historization & Slowly Changing Data

Tasks 81-95 • 15 practice tasks

#081 Why LEAD() for end-dates

INTERMEDIATE

In one sentence, explain why `LEAD() OVER (PARTITION BY prd_key ORDER BY prd_start_dt)` is the right tool to derive `prd_end_dt`, then write the query.

SOLUTION

```
SELECT prd_key, prd_start_dt, LEAD(prd_start_dt) OVER (PARTITION BY prd_key ORDER BY prd_start_dt) AS next_start_dt FROM silver.crm_prd_info;
```

- `LEAD()` reaches into the next chronological row of the same product without a self-join, which is exactly the shape of an effective-end-date rule.

#082 List historical product versions

BEGINNER

Return all rows in `silver.crm_prd_info` that represent a superseded (non-current) version, i.e. `prd_end_dt IS NOT NULL`.

SOLUTION

```
SELECT * FROM silver.crm_prd_info WHERE prd_end_dt IS NOT NULL;
```

- Historical rows matter for point-in-time reporting even though the gold dimension filters them out.

#083 List current product versions

BEGINNER

Return only the current version of each product (`prd_end_dt IS NULL`).

SOLUTION

```
SELECT * FROM silver.crm_prd_info WHERE prd_end_dt IS NULL;
```

- This is the exact `WHERE` clause used by `gold.dim_products` to keep the dimension at one row per product.

#084 Point-in-time price lookup

ADVANCED

Given an order date, find the product cost that was effective on that date (i.e. the version whose start/end range contains it).

SOLUTION

```
SELECT s.sls_ord_num, s.sls_order_dt, p.prđ_key, p.prđ_cost
FROM silver.crm_sales_details s
JOIN silver.crm_prđ_info p
  ON s.sls_prđ_key = p.prđ_key
 AND s.sls_order_dt >= p.prđ_start_dt
 AND (s.sls_order_dt <= p.prđ_end_dt OR p.prđ_end_dt IS NULL);
```

- This 'effective-dated join' pattern is the whole point of maintaining start/end dates on a dimension.

#085 Products with multiple versions

INTERMEDIATE

Count how many distinct prđ_key values have more than one historical row.

SOLUTION

```
SELECT prđ_key, COUNT(*) AS versions FROM silver.crm_prđ_info GROUP BY prđ_key HAVING COUNT(*) > 1;
```

- Products with several versions are the ones where a point-in-time join actually matters for correctness.

#086 Cost changes between versions

ADVANCED

For products with multiple versions, find cases where prđ_cost changed from one version to the next.

SOLUTION

```
SELECT prđ_key, prđ_start_dt, prđ_cost,
       LAG(prđ_cost) OVER (PARTITION BY prđ_key ORDER BY prđ_start_dt) AS prev_cost
FROM silver.crm_prđ_info;
-- filter in an outer query: WHERE prđ_cost <> prev_cost
```

- LAG() is the mirror image of LEAD() — useful for comparing a row to what came immediately before it.

#087 Version active-duration

INTERMEDIATE

Compute the number of days each product version was active (`prd_end_dt - prd_start_dt`), treating NULL end dates as still-active.

SOLUTION

```
SELECT prd_key, prd_start_dt, prd_end_dt,
       DATEDIFF(DAY, prd_start_dt, ISNULL(prd_end_dt, GETDATE())) AS days_active
FROM silver.crm_prd_info;
```

- Using `GETDATE()` as a stand-in for an open-ended NULL end date is a common historization convention.

#088 Detect overlapping version ranges

ADVANCED

Write a self-join to detect any two versions of the same product whose date ranges overlap, which would indicate a bug in the end-date derivation.

SOLUTION

```
SELECT a.prd_key, a.prd_start_dt, a.prd_end_dt, b.prd_start_dt AS b_start, b.prd_end_dt AS b_end
FROM silver.crm_prd_info a
JOIN silver.crm_prd_info b
  ON a.prd_key = b.prd_key AND a.prd_id <> b.prd_id
  AND a.prd_start_dt <= ISNULL(b.prd_end_dt, '9999-12-31')
  AND ISNULL(a.prd_end_dt, '9999-12-31') >= b.prd_start_dt;
```

- An empty result confirms the `LEAD()`-derived ranges never overlap — a good regression test to keep around.

#089 Detect gaps between versions

ADVANCED

Find cases where a gap exists between one version's end date + 1 day and the next version's start date.

SOLUTION

```
SELECT prd_key, prd_end_dt,
       LEAD(prd_start_dt) OVER (PARTITION BY prd_key ORDER BY prd_start_dt) AS next_start
FROM silver.crm_prd_info
-- filter in an outer query: WHERE DATEADD(DAY,1,prd_end_dt) <> next_start AND next_start IS NOT NULL
```

- With this pipeline's `LEAD()`-minus-one-day logic, gaps should never occur — this query is a useful sanity test.

#090 Alternative end-date via correlated subquery

INTERMEDIATE

Rewrite the `prd_end_dt` derivation using a correlated subquery instead of `LEAD()`, and compare readability.

SOLUTION

```
SELECT prd_id, prd_key, prd_start_dt,
       (SELECT MIN(p2.pr_d_start_dt) - 1
        FROM silver.crm_prd_info p2
        WHERE p2.pr_d_key = p1.pr_d_key AND p2.pr_d_start_dt > p1.pr_d_start_dt) AS prd_end_dt
FROM silver.crm_prd_info p1;
```

- Window functions are generally preferred here — the correlated subquery re-scans the table per row and doesn't scale as well.

#091 Full version history for one product

BEGINNER

Return the complete version history of a single `prd_key`, ordered chronologically.

SOLUTION

```
SELECT * FROM silver.crm_prd_info WHERE prd_key = 'FR-R92B-58' ORDER BY prd_start_dt;
```

- Pulling one entity's full history is a standard debugging step when a point-in-time join looks wrong.

#092 First and latest version per product

BEGINNER

For each `prd_key`, return its first and its most recent `prd_start_dt`.

SOLUTION

```
SELECT prd_key, MIN(prd_start_dt) AS first_version, MAX(prd_start_dt) AS latest_version FROM silver.crm_prd_info GROUP BY prd_key;
```

- MIN/MAX with GROUP BY is often simpler than a window function when you only need the two extremes.

#093 Products with only one version

BEGINNER

Find products that have never changed — exactly one row in `silver.crm_prd_info`.

SOLUTION

```
SELECT prd_key FROM silver.crm_prd_info GROUP BY prd_key HAVING COUNT(*) = 1;
```

- Single-version products are the majority in most catalogs; historization mainly matters for the minority that change.

#094 Point-in-time fact join into gold

ADVANCED

Modify the fact_sales join logic so that instead of the current dim_products row, each sale joins to the product version that was active on its order date.

SOLUTION

```
SELECT sd.sls_ord_num, sd.sls_order_dt, pn.prđ_cost AS effective_cost
FROM silver.crm_sales_details sd
JOIN silver.crm_prđ_info pn
  ON sd.sls_prđ_key = pn.prđ_key
  AND sd.sls_order_dt >= pn.prđ_start_dt
  AND (sd.sls_order_dt <= pn.prđ_end_dt OR pn.prđ_end_dt IS NULL);
```

- This is what a true Type 2 SCD fact join looks like, versus fact_sales' simpler always-join-to-current design.

#095 Count currently active versions

BEGINNER

Count how many product versions in silver.crm_prđ_info are currently active today.

SOLUTION

```
SELECT COUNT(*) FROM silver.crm_prđ_info WHERE prđ_end_dt IS NULL OR prđ_end_dt >= CAST(GETDATE() AS DATE);
```

- This should equal the row count of gold.dim_products, since that view filters on the same condition.

E. Cross-Source Integration (CRM + ERP)

Tasks 96–115 • 20 practice tasks

#096 CRM customers missing from ERP

INTERMEDIATE

Find customers in silver.crm_cust_info that have no matching row in silver.erp_cust_az12.

SOLUTION

```
SELECT ci.* FROM silver.crm_cust_info ci
LEFT JOIN silver.erp_cust_az12 ca ON ci.cst_key = ca.cid
WHERE ca.cid IS NULL;
```

- A LEFT JOIN with a NULL check on the right side is the standard anti-join pattern.

#097 ERP customers missing from CRM

INTERMEDIATE

Find customers in silver.erp_cust_az12 that have no matching row in silver.crm_cust_info.

SOLUTION

```
SELECT ca.* FROM silver.erp_cust_az12 ca
LEFT JOIN silver.crm_cust_info ci ON ca.cid = ci.cst_key
WHERE ci.cst_key IS NULL;
```

- Running the anti-join in both directions gives a complete picture of source coverage gaps.

#098 Customers missing country data

BEGINNER

Find customers in silver.crm_cust_info with no matching row in silver.erp_loc_a101.

SOLUTION

```
SELECT ci.cst_id, ci.cst_key FROM silver.crm_cust_info ci
LEFT JOIN silver.erp_loc_a101 la ON ci.cst_key = la.cid
WHERE la.cid IS NULL;
```

- These customers will show 'n/a' country in gold.dim_customers via the COALESCE/LEFT JOIN default.

#099 Key-format consistency across sources

INTERMEDIATE

Verify that all silver.crm_cust_info.cst_key values follow the same character-length pattern as silver.erp_cust_az12.cid after cleaning.

SOLUTION

```
SELECT DISTINCT LEN(cst_key) FROM silver.crm_cust_info EXCEPT SELECT DISTINCT LEN(cid) FROM silver.erp_cust_az12;
```

- An empty result confirms both cleaned key sets share a consistent format, which is a prerequisite for reliable joins.

#100 Gender mismatches between CRM and ERP

INTERMEDIATE

Find customers where cst_gndr (CRM) and gen (ERP) disagree, excluding cases where either source says 'n/a'.

SOLUTION

```
SELECT ci.cst_key, ci.cst_gndr, ca.gen
FROM silver.crm_cust_info ci
JOIN silver.erp_cust_az12 ca ON ci.cst_key = ca.cid
WHERE ci.cst_gndr <> ca.gen AND ci.cst_gndr <> 'n/a' AND ca.gen <> 'n/a';
```

- This mismatch report justifies gold.dim_customers' rule of trusting CRM first and only falling back to ERP.

#101 Customer count by country

BEGINNER

Count customers per country by joining crm_cust_info to erp_loc_a101.

SOLUTION

```
SELECT la.cntry, COUNT(*) AS customer_count
FROM silver.crm_cust_info ci
JOIN silver.erp_loc_a101 la ON ci.cst_key = la.cid
GROUP BY la.cntry
ORDER BY customer_count DESC;
```

- A simple join + GROUP BY is often all that's needed once the keys are already clean.

#102 Products with an unknown category

INTERMEDIATE

Find products in silver.crm_prd_info whose cat_id has no match in silver.erp_px_cat_glv2.

SOLUTION

```
SELECT pn.prd_key, pn.cat_id
FROM silver.crm_prd_info pn
LEFT JOIN silver.erp_px_cat_glv2 pc ON pn.cat_id = pc.id
WHERE pc.id IS NULL;
```

- These products will show a NULL category/subcategory in gold.dim_products due to the LEFT JOIN.

#103 Unused categories

BEGINNER

Find categories in silver.erp_px_cat_glv2 that have no matching products.

SOLUTION

```
SELECT pc.* FROM silver.erp_px_cat_glv2 pc
LEFT JOIN silver.crm_prd_info pn ON pc.id = pn.cat_id
WHERE pn.cat_id IS NULL;
```

- Unused reference/lookup rows are common and usually harmless, but worth a periodic check for stale data.

#104 Full outer join of customer sources

INTERMEDIATE

Use a FULL OUTER JOIN between crm_cust_info and erp_cust_az12 to see the complete union of both sources, flagging which side each row came from.

SOLUTION

```
SELECT
  COALESCE(ci.cst_key, ca.cid) AS key_value,
  CASE WHEN ci.cst_key IS NULL THEN 'ERP only'
        WHEN ca.cid IS NULL THEN 'CRM only'
        ELSE 'Both' END AS source_coverage
FROM silver.crm_cust_info ci
FULL OUTER JOIN silver.erp_cust_az12 ca ON ci.cst_key = ca.cid;
```

- A FULL OUTER JOIN with a coverage flag is a great one-query summary of two sources' overlap.

#105 Orphan product keys in sales

INTERMEDIATE

Find `sls_prd_key` values in `silver.crm_sales_details` that have no matching row in `silver.crm_prd_info`.

SOLUTION

```
SELECT DISTINCT sd.sls_prd_key
FROM silver.crm_sales_details sd
LEFT JOIN silver.crm_prd_info pn ON sd.sls_prd_key = pn.prn_key
WHERE pn.prn_key IS NULL;
```

- Orphan foreign keys in a transaction table are exactly what will produce NULL `product_key` rows in `gold.fact_sales`.

#106 Orphan customer ids in sales

INTERMEDIATE

Find `sls_cust_id` values in `silver.crm_sales_details` that have no matching row in `silver.crm_cust_info`.

SOLUTION

```
SELECT DISTINCT sd.sls_cust_id
FROM silver.crm_sales_details sd
LEFT JOIN silver.crm_cust_info ci ON sd.sls_cust_id = ci.cst_id
WHERE ci.cst_id IS NULL;
```

- Same idea as the product check — orphan customer references become NULL `customer_key` rows in the fact view.

#107 Customer key reconciliation table

ADVANCED

Build a reconciliation result set showing each distinct customer key, which source(s) it appears in, and a `match_status` flag.

SOLUTION

```
SELECT COALESCE(ci.cst_key, ca.cid, la.cid) AS customer_key,
       CASE WHEN ci.cst_key IS NULL THEN 'Missing CRM' ELSE 'OK' END AS crm_status,
       CASE WHEN ca.cid IS NULL THEN 'Missing ERP demo' ELSE 'OK' END AS erp_demo_status,
       CASE WHEN la.cid IS NULL THEN 'Missing ERP location' ELSE 'OK' END AS erp_loc_status
FROM silver.crm_cust_info ci
FULL OUTER JOIN silver.erp_cust_az12 ca ON ci.cst_key = ca.cid
FULL OUTER JOIN silver.erp_loc_a101 la ON COALESCE(ci.cst_key, ca.cid) = la.cid;
```

- A three-way reconciliation view is a useful one-off audit artifact when onboarding a new source system.

#108 Match rate between CRM and ERP

BEGINNER

Calculate the percentage of crm_cust_info customers that successfully match to erp_cust_az12.

SOLUTION

```
SELECT
  CAST(SUM(CASE WHEN ca.cid IS NOT NULL THEN 1 ELSE 0 END) AS FLOAT) / COUNT(*) * 100 AS match_pct
FROM silver.crm_cust_info ci
LEFT JOIN silver.erp_cust_az12 ca ON ci.cst_key = ca.cid;
```

- A single match-rate KPI is a quick way to monitor cross-source join health over time.

#109 Duplicate keys that would break a join

INTERMEDIATE

Find any cid values that appear more than once in silver.erp_loc_a101, which would cause row fan-out on a join.

SOLUTION

```
SELECT cid, COUNT(*) FROM silver.erp_loc_a101 GROUP BY cid HAVING COUNT(*) > 1;
```

- A one-to-many join where you expect one-to-one silently inflates row counts — always check for this.

#110 Duplicate category ids

BEGINNER

Find any id values that appear more than once in silver.erp_px_cat_g1v2.

SOLUTION

```
SELECT id, COUNT(*) FROM silver.erp_px_cat_g1v2 GROUP BY id HAVING COUNT(*) > 1;
```

- The same fan-out risk applies to the product-category lookup used in gold.dim_products.

#111 Reproduce the dim_customers gender-fallback join

ADVANCED

Write the standalone query that CRM-first, ERP-fallback gender logic used by gold.dim_customers, without using the view.

SOLUTION

```
SELECT ci.cst_id,  
       CASE WHEN ci.cst_gndr <> 'n/a' THEN ci.cst_gndr ELSE COALESCE(ca.gen, 'n/a') END AS gender  
FROM silver.crm_cust_info ci  
LEFT JOIN silver.erp_cust_az12 ca ON ci.cst_key = ca.cid;
```

- Reproducing a view's logic by hand is a great way to confirm you understand exactly what it does.

#112 Reproduce the dim_products category join

INTERMEDIATE

Write the standalone query joining crm_prd_info to erp_px_cat_g1v2 via cat_id, as used by gold.dim_products.

SOLUTION

```
SELECT pn.prd_key, pn.prd_nm, pc.cat, pc.subcat, pc.maintenance  
FROM silver.crm_prd_info pn  
LEFT JOIN silver.erp_px_cat_g1v2 pc ON pn.cat_id = pc.id  
WHERE pn.prd_end_dt IS NULL;
```

- Note the WHERE clause on prd_end_dt — it's what keeps the dimension at the current-version grain.

#113 Inconsistent ERP demographic duplicates

INTERMEDIATE

Check whether any cid in erp_cust_az12 has more than one row with a different bdate or gen (a data-quality red flag, not just a duplicate key).

SOLUTION

```
SELECT cid, COUNT(DISTINCT bdate) AS distinct_bdates, COUNT(DISTINCT gen) AS distinct_gens  
FROM silver.erp_cust_az12  
GROUP BY cid  
HAVING COUNT(DISTINCT bdate) > 1 OR COUNT(DISTINCT gen) > 1;
```

- A duplicate key with conflicting attribute values is worse than a simple duplicate — it means you can't just pick 'any' row.

#114 CRM customers with zero sales

BEGINNER

Count how many silver.crm_cust_info customers have never placed an order.

SOLUTION

```
SELECT COUNT(*) FROM silver.crm_cust_info ci
LEFT JOIN silver.crm_sales_details sd ON ci.cst_id = sd.sls_cust_id
WHERE sd.sls_cust_id IS NULL;
```

- This is a common 'never purchased' segment used for marketing or churn-risk reporting.

#115 ERP customers with zero sales

BEGINNER

Count how many silver.erp_cust_az12 customers have never placed an order (joining through crm_cust_info to reach sales).

SOLUTION

```
SELECT COUNT(*) FROM silver.erp_cust_az12 ca
JOIN silver.crm_cust_info ci ON ca.cid = ci.cst_key
LEFT JOIN silver.crm_sales_details sd ON ci.cst_id = sd.sls_cust_id
WHERE sd.sls_cust_id IS NULL;
```

- Because sales only carries the CRM customer id, reaching it from the ERP side always requires a bridging join through CRM.

F. Gold Layer — Dimensional Modeling

Tasks 116-140 • 25 practice tasks

#116 Create gold.dim_customers

INTERMEDIATE

Create the gold.dim_customers view joining CRM as the base with ERP demographic and location data, generating a surrogate key.

SOLUTION

```
CREATE VIEW gold.dim_customers AS
SELECT
  ROW_NUMBER() OVER (ORDER BY cst_id) AS customer_key,
  ci.cst_id AS customer_id, ci.cst_key AS customer_number,
  ci.cst_firstname AS first_name, ci.cst_lastname AS last_name,
  la.centry AS country, ci.cst_marital_status AS marital_status,
  CASE WHEN ci.cst_gndr <> 'n/a' THEN ci.cst_gndr ELSE COALESCE(ca.gen, 'n/a') END AS gender,
  ca.bdate AS birthdate, ci.cst_create_date AS create_date
FROM silver.crm_cust_info ci
LEFT JOIN silver.erp_cust_az12 ca ON ci.cst_key = ca.cid
LEFT JOIN silver.erp_loc_a101 la ON ci.cst_key = la.cid;
```

- ROW_NUMBER() OVER (ORDER BY cst_id) generates a stable surrogate key independent of the natural business key.

#117 Create gold.dim_products

INTERMEDIATE

Create the gold.dim_products view, restricted to current-version products, with a surrogate key.

SOLUTION

```
CREATE VIEW gold.dim_products AS
SELECT
  ROW_NUMBER() OVER (ORDER BY pn.prd_start_dt, pn.prd_key) AS product_key,
  pn.prd_id AS product_id, pn.prd_key AS product_number, pn.prd_nm AS product_name,
  pn.cat_id AS category_id, pc.cat AS category, pc.subcat AS subcategory, pc.maintenance,
  pn.prd_cost AS cost, pn.prd_line AS product_line, pn.prd_start_dt AS start_date
FROM silver.crm_prd_info pn
LEFT JOIN silver.erp_px_cat_glv2 pc ON pn.cat_id = pc.id
WHERE pn.prd_end_dt IS NULL;
```

- Filtering to prd_end_dt IS NULL before generating the surrogate key keeps the dimension at exactly one row per current product.

#118 Why filter on prd_end_dt IS NULL?

BEGINNER

Explain and verify with a query why gold.dim_products must filter to prd_end_dt IS NULL to preserve a one-row-per-product grain.

SOLUTION

```
SELECT prd_key, COUNT(*) FROM silver.crm_prd_info WHERE prd_end_dt IS NULL GROUP BY prd_key HAVING COUNT(*) > 1;
```

- An empty result proves the filter is sufficient — without it, historical rows would duplicate every product.

#119 Verify dim_customers grain

BEGINNER

Verify gold.dim_customers has exactly one row per customer_id.

SOLUTION

```
SELECT customer_id, COUNT(*) FROM gold.dim_customers GROUP BY customer_id HAVING COUNT(*) > 1;
```

- A grain-violation query like this should be run on every dimension after any change to its join logic.

#120 Verify dim_products grain

BEGINNER

Verify gold.dim_products has exactly one row per product_number.

SOLUTION

```
SELECT product_number, COUNT(*) FROM gold.dim_products GROUP BY product_number HAVING COUNT(*) > 1;
```

- Fan-out from a lookup join (e.g. duplicate category ids) would show up here as a failing row.

#121 Surrogate key uniqueness — customers

BEGINNER

Verify customer_key values in gold.dim_customers are unique.

SOLUTION

```
SELECT customer_key, COUNT(*) FROM gold.dim_customers GROUP BY customer_key HAVING COUNT(*) > 1;
```

- ROW_NUMBER() guarantees uniqueness by construction, but it's still worth testing after any view change.

#122 Surrogate key uniqueness — products

BEGINNER

Verify product_key values in gold.dim_products are unique.

SOLUTION

```
SELECT product_key, COUNT(*) FROM gold.dim_products GROUP BY product_key HAVING COUNT(*) > 1;
```

- Same reasoning as the customer surrogate key check — cheap insurance against a broken view definition.

#123 Derive customer age

INTERMEDIATE

Add a derived age column to gold.dim_customers based on birthdate.

SOLUTION

```
SELECT customer_id, birthdate, DATEDIFF(YEAR, birthdate, GETDATE()) AS age FROM gold.dim_customers;
```

- DATEDIFF(YEAR, ...) is an approximation — it doesn't account for whether the birthday has occurred yet this year.

#124 Derive customer tenure

INTERMEDIATE

Add a derived tenure-in-years column to gold.dim_customers based on create_date.

SOLUTION

```
SELECT customer_id, create_date, DATEDIFF(YEAR, create_date, GETDATE()) AS tenure_years FROM gold.dim_customers;
```

- Tenure is a common input feature for churn models and loyalty segmentation.

#125 Cost tier bucket for products

INTERMEDIATE

Add a derived cost_tier column to gold.dim_products: Low (<50), Medium (50-499), High (500+).

SOLUTION

```
SELECT product_number, cost,
       CASE WHEN cost < 50 THEN 'Low' WHEN cost < 500 THEN 'Medium' ELSE 'High' END AS cost_tier
FROM gold.dim_products;
```

- Bucketing a continuous measure into a categorical attribute is a common gold-layer enrichment for BI filtering.

#126 Reverse the gender priority

ADVANCED

Rebuild the gender logic to prefer ERP over CRM instead, and compare how many rows change versus the original view.

SOLUTION

```
SELECT ci.cst_id,  
       CASE WHEN ca.gen <> 'n/a' THEN ca.gen ELSE COALESCE(ci.cst_gndr, 'n/a') END AS gender_erp_first  
FROM silver.crm_cust_info ci  
LEFT JOIN silver.erp_cust_az12 ca ON ci.cst_key = ca.cid;  
-- compare row-by-row against gold.dim_customers.gender to count differences
```

- Testing an alternative business rule side-by-side is a good way to justify (or challenge) the original design choice.

#127 Sanity-check country values

BEGINNER

List the distinct country values exposed by gold.dim_customers to confirm no leftover raw codes remain.

SOLUTION

```
SELECT DISTINCT country FROM gold.dim_customers ORDER BY country;
```

- If any two-letter code appears here, the silver-layer country normalization missed a case.

#128 Bonus gold.dim_country lookup

ADVANCED

Design and create a slim gold.dim_country view exposing just the distinct country values as a reusable lookup.

SOLUTION

```
CREATE VIEW gold.dim_country AS SELECT DISTINCT country FROM gold.dim_customers WHERE country IS NOT NULL;
```

- Small conformed dimensions like this are useful when multiple fact tables need to filter or slice by the same attribute.

#129 NULL surrogate key check

BEGINNER

Verify no NULL customer_key or product_key values exist in either gold dimension.

SOLUTION

```
SELECT COUNT(*) FROM gold.dim_customers WHERE customer_key IS NULL  
UNION ALL  
SELECT COUNT(*) FROM gold.dim_products WHERE product_key IS NULL;
```

- ROW_NUMBER() can never produce NULL, so any non-zero count here indicates something else broke the view.

#130 Document dim_customers columns

BEGINNER

Produce a business-friendly column listing for gold.dim_customers using INFORMATION_SCHEMA, ready to paste into a data dictionary.

SOLUTION

```
SELECT COLUMN_NAME, DATA_TYPE
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = 'gold' AND TABLE_NAME = 'dim_customers'
ORDER BY ORDINAL_POSITION;
```

- Querying the system catalog keeps a data dictionary accurate without manually re-typing column names.

#131 Document dim_products columns

BEGINNER

Produce the same INFORMATION_SCHEMA column listing for gold.dim_products.

SOLUTION

```
SELECT COLUMN_NAME, DATA_TYPE
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = 'gold' AND TABLE_NAME = 'dim_products'
ORDER BY ORDINAL_POSITION;
```

- Automated documentation queries are worth keeping in a shared 'catalog' script for the whole project.

#132 Detect a product-number grain violation

ADVANCED

Write a query that would detect if any product_number ever mapped to two different product_key values (a grain violation, even though the current view design prevents it).

SOLUTION

```
SELECT product_number, COUNT(DISTINCT product_key) AS key_variants
FROM gold.dim_products
GROUP BY product_number
HAVING COUNT(DISTINCT product_key) > 1;
```

- A defensive test like this costs nothing to run and catches regressions if the view is ever refactored.

#133 Row-count reconciliation: dim_customers

INTERMEDIATE

Compare the row count of gold.dim_customers against the distinct cst_id count in silver.crm_cust_info and explain any gap.

SOLUTION

```
SELECT (SELECT COUNT(*) FROM gold.dim_customers) AS dim_rows,  
       (SELECT COUNT(DISTINCT cst_id) FROM silver.crm_cust_info) AS silver_distinct_customers;
```

- These two numbers should match exactly, since dim_customers' base table is already deduplicated in silver.

#134 Row-count reconciliation: dim_products

INTERMEDIATE

Compare gold.dim_products' row count against the count of current-version rows in silver.crm_prd_info.

SOLUTION

```
SELECT (SELECT COUNT(*) FROM gold.dim_products) AS dim_rows,  
       (SELECT COUNT(*) FROM silver.crm_prd_info WHERE prd_end_dt IS NULL) AS silver_current_products;
```

- Any mismatch here points to either a missing category-join fan-out or a grain assumption bug in the view.

#135 Type-2 style historical product view

ADVANCED

Design a separate gold.dim_products_history view that, unlike gold.dim_products, exposes every historical version with valid_from/valid_to columns.

SOLUTION

```
CREATE VIEW gold.dim_products_history AS  
SELECT  
  ROW_NUMBER() OVER (ORDER BY pn.prd_key, pn.prd_start_dt) AS product_version_key,  
  pn.prd_key AS product_number, pn.prd_nm AS product_name,  
  pn.prd_cost AS cost, pn.prd_start_dt AS valid_from, pn.prd_end_dt AS valid_to  
FROM silver.crm_prd_info pn;
```

- Keeping the current-state and full-history dimensions as two separate views is cleaner than overloading one view with both grains.

#136 Marital-status single flag

INTERMEDIATE

Add a derived `is_single` (Y/N) flag to `gold.dim_customers` based on `marital_status`.

SOLUTION

```
SELECT customer_id, marital_status, CASE WHEN marital_status = 'Single' THEN 'Y' ELSE 'N' END AS is_single
FROM gold.dim_customers;
```

- A boolean-style flag column is often easier for BI tools to filter on than a text comparison.

#137 Metadata header for `dim_customers`

BEGINNER

Write a documentation-style comment block (as would sit above the `CREATE VIEW` statement) describing `gold.dim_customers`' purpose, grain, and key columns.

SOLUTION

```
/*
=====
View: gold.dim_customers
Purpose: Customer dimension – one row per customer, combining CRM identity
        data with ERP demographic (gender/birthdate) and location data.
Grain:   One row per customer_id.
Key:     customer_key (surrogate, ROW_NUMBER over cst_id)
=====
*/
```

- A short header comment like this is standard practice for any view feeding a BI layer.

#138 PII review of gold dimensions

INTERMEDIATE

Review `gold.dim_customers`' columns and identify which should be restricted from general business-user access (PII), versus safe to expose broadly.

SOLUTION

```
SELECT COLUMN_NAME FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_SCHEMA = 'gold' AND TABLE_NAME =
'dim_customers';
```

- `first_name`, `last_name`, and `birthdate` are typical PII candidates; `customer_key`, `country`, and `gender` are usually fine for broad analytical access.

#139 Expose category_key on dim_products

INTERMEDIATE

Modify gold.dim_products to also expose erp_px_cat_glv2.id as category_key, alongside the existing category_id, for clearer downstream joins.

SOLUTION

```
SELECT pn.prk_key, pn.cat_id AS category_id, pc.id AS category_key, pc.cat AS category
FROM silver.crm_prd_info pn
LEFT JOIN silver.erp_px_cat_glv2 pc ON pn.cat_id = pc.id
WHERE pn.prk_end_dt IS NULL;
```

- Exposing both the source-system id and a clearly-named alias avoids ambiguity for anyone consuming the view later.

#140 Gold dimension smoke test script

ADVANCED

Write a single script that runs the grain, uniqueness, and NULL-key checks for both gold dimensions and prints PASS/FAIL for each.

SOLUTION

```
IF EXISTS (SELECT 1 FROM gold.dim_customers GROUP BY customer_id HAVING COUNT(*) > 1)
  PRINT 'FAIL: dim_customers grain violation'; ELSE PRINT 'PASS: dim_customers grain';
IF EXISTS (SELECT 1 FROM gold.dim_products GROUP BY product_number HAVING COUNT(*) > 1)
  PRINT 'FAIL: dim_products grain violation'; ELSE PRINT 'PASS: dim_products grain';
```

- A lightweight PASS/FAIL smoke test script is easy to run manually after any view change, even without a full test framework.

G. Gold Layer — Fact Table & Referential Integrity

Tasks 141-160 • 20 practice tasks

#141 Create gold.fact_sales

INTERMEDIATE

Create the gold.fact_sales view joining cleaned sales details to both gold dimensions via their business keys.

SOLUTION

```
CREATE VIEW gold.fact_sales AS
SELECT
    sd.sls_ord_num AS order_number, pr.product_key, cu.customer_key,
    sd.sls_order_dt AS order_date, sd.sls_ship_dt AS shipping_date, sd.sls_due_dt AS due_date,
    sd.sls_sales AS sales_amount, sd.sls_quantity AS quantity, sd.sls_price AS price
FROM silver.crm_sales_details sd
LEFT JOIN gold.dim_products pr ON sd.sls_prd_key = pr.product_number
LEFT JOIN gold.dim_customers cu ON sd.sls_cust_id = cu.customer_id;
```

- The fact view always joins to gold dimensions by business key, then exposes only the resulting surrogate keys.

#142 NULL customer_key check

INTERMEDIATE

Verify how many gold.fact_sales rows have a NULL customer_key (an unmatched customer).

SOLUTION

```
SELECT COUNT(*) FROM gold.fact_sales WHERE customer_key IS NULL;
```

- A LEFT JOIN preserves the sales row even without a dimension match, at the cost of a NULL surrogate key.

#143 NULL product_key check

INTERMEDIATE

Verify how many gold.fact_sales rows have a NULL product_key.

SOLUTION

```
SELECT COUNT(*) FROM gold.fact_sales WHERE product_key IS NULL;
```

- This count should match the orphan sls_prd_key values found earlier in the cross-source integration checks.

#144 Post-transform sales formula check

BEGINNER

Confirm the formula `sales_amount = quantity * price` still holds in `gold.fact_sales` after silver-layer cleaning.

SOLUTION

```
SELECT * FROM gold.fact_sales WHERE sales_amount <> quantity * price;
```

- An empty result here validates that the silver-layer recalculation rule worked as intended.

#145 Order date after shipping date

INTERMEDIATE

Find `gold.fact_sales` rows where `order_date` is later than `shipping_date`.

SOLUTION

```
SELECT * FROM gold.fact_sales WHERE order_date > shipping_date;
```

- Any rows returned suggest either a data quality issue or a system clock/timezone problem at the source.

#146 Order date after due date

BEGINNER

Find `gold.fact_sales` rows where `order_date` is later than `due_date`.

SOLUTION

```
SELECT * FROM gold.fact_sales WHERE order_date > due_date;
```

- A due date earlier than the order date would be a logical impossibility worth escalating to the source system owner.

#147 Revenue by product without a dimension join

BEGINNER

Sum `sales_amount` by `product_key` directly from `gold.fact_sales`, with no dimension join, as a base-grain sanity check.

SOLUTION

```
SELECT product_key, SUM(sales_amount) AS total_revenue FROM gold.fact_sales GROUP BY product_key ORDER BY total_revenue DESC;
```

- Aggregating the fact table alone (before joining dimensions) is a useful way to isolate fact-layer issues from dimension issues.

#148 Determine and verify fact grain

ADVANCED

State the grain of gold.fact_sales in one sentence, then write a query proving order_number + product_key is unique (or explain why it isn't).

SOLUTION

```
SELECT order_number, product_key, COUNT(*) FROM gold.fact_sales GROUP BY order_number, product_key HAVING COUNT(*) > 1;
```

- Grain: one row per order line item (order_number + product_key). Any duplicates returned mean the assumed grain is wrong.

#149 Multi-line orders

BEGINNER

Find order_number values that appear with more than one distinct product_key.

SOLUTION

```
SELECT order_number, COUNT(DISTINCT product_key) AS line_items FROM gold.fact_sales GROUP BY order_number HAVING COUNT(DISTINCT product_key) > 1;
```

- Multi-line orders are completely normal — this query is about understanding the data, not flagging a defect.

#150 Overall revenue, quantity, and average price

BEGINNER

Calculate total sales_amount, total quantity, and average price across all of gold.fact_sales.

SOLUTION

```
SELECT SUM(sales_amount) AS total_revenue, SUM(quantity) AS total_qty, AVG(price) AS avg_price FROM gold.fact_sales;
```

- A handful of headline aggregates like this are usually the first query run against any new fact table.

#151 Top 10 orders by revenue

BEGINNER

Find the 10 highest sales_amount rows in gold.fact_sales.

SOLUTION

```
SELECT TOP (10) order_number, product_key, sales_amount FROM gold.fact_sales ORDER BY sales_amount DESC;
```

- TOP (N) ... ORDER BY DESC is the standard SQL Server pattern for a simple leaderboard query.

#152 Missing order dates

BEGINNER

Find gold.fact_sales rows where order_date is NULL entirely.

SOLUTION

```
SELECT * FROM gold.fact_sales WHERE order_date IS NULL;
```

- These are the rows whose original 8-digit date failed the silver-layer validity check.

#153 Monthly sales summary

INTERMEDIATE

Summarize total sales_amount by calendar month across gold.fact_sales.

SOLUTION

```
SELECT DATEFROMPARTS(YEAR(order_date), MONTH(order_date), 1) AS sales_month, SUM(sales_amount) AS total_revenue
FROM gold.fact_sales
WHERE order_date IS NOT NULL
GROUP BY DATEFROMPARTS(YEAR(order_date), MONTH(order_date), 1)
ORDER BY sales_month;
```

- DATEFROMPARTS is a clean way to bucket dates to the first of the month without string manipulation.

#154 Fact vs. silver row-count parity

INTERMEDIATE

Verify gold.fact_sales has exactly the same row count as silver.crm_sales_details, confirming the LEFT JOINs never drop or duplicate rows.

SOLUTION

```
SELECT (SELECT COUNT(*) FROM gold.fact_sales) AS fact_rows,
       (SELECT COUNT(*) FROM silver.crm_sales_details) AS silver_rows;
```

- Because both dimension joins use LEFT JOIN on (expected) unique keys, the fact row count must equal the silver source row count.

#155 Full-row duplicate check on fact

INTERMEDIATE

Check for fully duplicated rows in gold.fact_sales (same order_number, product_key, and all measures).

SOLUTION

```
SELECT order_number, product_key, customer_key, order_date, sales_amount, quantity, price, COUNT(*)
FROM gold.fact_sales
GROUP BY order_number, product_key, customer_key, order_date, sales_amount, quantity, price
HAVING COUNT(*) > 1;
```

- Full-row duplicates in a fact view usually trace back to a fan-out from a dimension join, not the source data itself.

#156 Incremental fact extract by date range

ADVANCED

Write a parameterized version of the fact_sales query limited to orders within a given date range, suitable for an incremental extract.

SOLUTION

```
DECLARE @start_date DATE = '2013-01-01', @end_date DATE = '2013-12-31';
SELECT * FROM gold.fact_sales
WHERE order_date BETWEEN @start_date AND @end_date;
```

- Even though the underlying view is full-refresh, downstream consumers can still extract incrementally using a date filter.

#157 Unmatched product percentage

INTERMEDIATE

Calculate the percentage of gold.fact_sales rows with a NULL product_key.

SOLUTION

```
SELECT CAST(SUM(CASE WHEN product_key IS NULL THEN 1 ELSE 0 END) AS FLOAT) / COUNT(*) * 100 AS pct_unmatched_products
FROM gold.fact_sales;
```

- Tracking this percentage over time turns an ad-hoc DQ check into a trend you can alert on.

#158 Unmatched customer percentage

INTERMEDIATE

Calculate the percentage of gold.fact_sales rows with a NULL customer_key.

SOLUTION

```
SELECT CAST(SUM(CASE WHEN customer_key IS NULL THEN 1 ELSE 0 END) AS FLOAT) / COUNT(*) * 100 AS pct_unmatched_customers FROM gold.fact_sales;
```

- A rising trend in this metric over successive loads would indicate a growing upstream data quality problem.

#159 Shipping lead time

BEGINNER

Add a derived lead_time_days column to gold.fact_sales measuring days between order_date and shipping_date.

SOLUTION

```
SELECT order_number, DATEDIFF(DAY, order_date, shipping_date) AS lead_time_days FROM gold.fact_sales;
```

- Lead-time analysis is one of the most common operational metrics built directly on top of a sales fact table.

#160 Flag late shipments

INTERMEDIATE

Flag orders where shipping_date is later than due_date.

SOLUTION

```
SELECT order_number, shipping_date, due_date, CASE WHEN shipping_date > due_date THEN 'Late' ELSE 'On Time' END AS shipment_status FROM gold.fact_sales;
```

- A simple CASE-based status flag is often more useful to business users than the raw date columns alone.

H. Analytics & Aggregation on the Gold Layer

Tasks 161-185 • 25 practice tasks

#161 Total sales by country

BEGINNER

Total sales_amount by customer country.

SOLUTION

```
SELECT c.country, SUM(f.sales_amount) AS total_sales
FROM gold.fact_sales f JOIN gold.dim_customers c ON f.customer_key = c.customer_key
GROUP BY c.country ORDER BY total_sales DESC;
```

- Joining the fact table to a dimension to slice by an attribute is the most fundamental star-schema query pattern.

#162 Total sales by category

BEGINNER

Total sales_amount by product category.

SOLUTION

```
SELECT p.category, SUM(f.sales_amount) AS total_sales
FROM gold.fact_sales f JOIN gold.dim_products p ON f.product_key = p.product_key
GROUP BY p.category ORDER BY total_sales DESC;
```

- Category-level revenue is usually the first slice a business stakeholder asks for.

#163 Total sales by subcategory

BEGINNER

Total sales_amount by product subcategory, sorted descending.

SOLUTION

```
SELECT p.subcategory, SUM(f.sales_amount) AS total_sales
FROM gold.fact_sales f JOIN gold.dim_products p ON f.product_key = p.product_key
GROUP BY p.subcategory ORDER BY total_sales DESC;
```

- Subcategory adds one more level of drill-down granularity beneath category.

#164 Monthly sales trend

INTERMEDIATE

Build a year-month sales trend from gold.fact_sales.

SOLUTION

```
SELECT YEAR(order_date) AS yr, MONTH(order_date) AS mo, SUM(sales_amount) AS total_sales
FROM gold.fact_sales WHERE order_date IS NOT NULL
GROUP BY YEAR(order_date), MONTH(order_date)
ORDER BY yr, mo;
```

- Grouping by YEAR() and MONTH() separately keeps the trend easy to chart without extra date formatting.

#165 Yearly sales with YoY growth

INTERMEDIATE

Compute yearly total sales and the year-over-year growth percentage.

SOLUTION

```
WITH yearly AS (
  SELECT YEAR(order_date) AS yr, SUM(sales_amount) AS total_sales
  FROM gold.fact_sales WHERE order_date IS NOT NULL GROUP BY YEAR(order_date)
)
SELECT yr, total_sales,
  LAG(total_sales) OVER (ORDER BY yr) AS prev_year_sales,
  (total_sales - LAG(total_sales) OVER (ORDER BY yr)) * 100.0 / LAG(total_sales) OVER (ORDER BY yr) AS yoy_growth_pct
FROM yearly;
```

- A CTE followed by LAG() is the standard pattern for period-over-period growth calculations.

#166 Top 10 customers by spend

INTERMEDIATE

Find the top 10 customers by total sales_amount.

SOLUTION

```
SELECT TOP (10) c.customer_id, c.first_name, c.last_name, SUM(f.sales_amount) AS total_spend
FROM gold.fact_sales f JOIN gold.dim_customers c ON f.customer_key = c.customer_key
GROUP BY c.customer_id, c.first_name, c.last_name
ORDER BY total_spend DESC;
```

- A top-customer leaderboard is a common input to VIP or loyalty-program targeting.

#167 Top 5 products by revenue

BEGINNER

Find the top 5 products by total revenue.

SOLUTION

```
SELECT TOP (5) p.product_name, SUM(f.sales_amount) AS total_revenue
FROM gold.fact_sales f JOIN gold.dim_products p ON f.product_key = p.product_key
GROUP BY p.product_name ORDER BY total_revenue DESC;
```

- Revenue and unit-volume leaderboards (see the next task) often surface different 'best' products.

#168 Top 5 products by quantity sold

BEGINNER

Find the top 5 products by total quantity sold.

SOLUTION

```
SELECT TOP (5) p.product_name, SUM(f.quantity) AS total_qty
FROM gold.fact_sales f JOIN gold.dim_products p ON f.product_key = p.product_key
GROUP BY p.product_name ORDER BY total_qty DESC;
```

- A high-volume, low-price product can rank very differently here versus the revenue leaderboard.

#169 Overall average order value

BEGINNER

Calculate the average sales_amount per order_number (order-level, not line-level).

SOLUTION

```
SELECT AVG(order_total) AS avg_order_value FROM (
    SELECT order_number, SUM(sales_amount) AS order_total FROM gold.fact_sales GROUP BY order_number
) t;
```

- Averaging line-level sales_amount directly would understate order value for multi-line orders — aggregate to order level first.

#170 Average order value by country

INTERMEDIATE

Calculate average order value broken out by customer country.

SOLUTION

```
SELECT c.country, AVG(order_total) AS avg_order_value
FROM (
    SELECT f.customer_key, f.order_number, SUM(f.sales_amount) AS order_total
    FROM gold.fact_sales f GROUP BY f.customer_key, f.order_number
) o
JOIN gold.dim_customers c ON o.customer_key = c.customer_key
GROUP BY c.country ORDER BY avg_order_value DESC;
```

- Building the order-level subquery first, then joining to the dimension, avoids double-counting multi-line orders.

#171 Running total of sales over time

INTERMEDIATE

Compute a running (cumulative) total of sales_amount ordered by order_date.

SOLUTION

```
SELECT order_date, sales_amount,
    SUM(sales_amount) OVER (ORDER BY order_date ROWS UNBOUNDED PRECEDING) AS running_total
FROM gold.fact_sales WHERE order_date IS NOT NULL
ORDER BY order_date;
```

- A window SUM with an ORDER BY frame is the standard way to compute a running total without a self-join.

#172 Rank products within category

INTERMEDIATE

Rank products by revenue within each category using RANK().

SOLUTION

```
SELECT p.category, p.product_name, SUM(f.sales_amount) AS revenue,
    RANK() OVER (PARTITION BY p.category ORDER BY SUM(f.sales_amount) DESC) AS category_rank
FROM gold.fact_sales f JOIN gold.dim_products p ON f.product_key = p.product_key
GROUP BY p.category, p.product_name;
```

- PARTITION BY resets the ranking for each category, so every category gets its own #1 product.

#173 Customer recency

ADVANCED

Calculate days since each customer's most recent order (recency, the 'R' in RFM).

SOLUTION

```
SELECT customer_key, DATEDIFF(DAY, MAX(order_date), GETDATE()) AS recency_days
FROM gold.fact_sales WHERE order_date IS NOT NULL
GROUP BY customer_key;
```

- Recency is usually the single strongest predictor of near-term repeat purchase likelihood.

#174 Customer frequency

INTERMEDIATE

Calculate each customer's distinct order count (frequency, the 'F' in RFM).

SOLUTION

```
SELECT customer_key, COUNT(DISTINCT order_number) AS order_frequency FROM gold.fact_sales GROUP BY
customer_key;
```

- COUNT(DISTINCT order_number) rather than COUNT(*) avoids over-counting customers with multi-line orders.

#175 Customer monetary value

INTERMEDIATE

Calculate each customer's total spend (monetary, the 'M' in RFM).

SOLUTION

```
SELECT customer_key, SUM(sales_amount) AS total_spend FROM gold.fact_sales GROUP BY customer_key;
```

- Combine this with the recency and frequency tasks above to build a full three-metric RFM table.

#176 Month-over-month sales change

INTERMEDIATE

Calculate the month-over-month change in total sales using LAG().

SOLUTION

```
WITH monthly AS (  
    SELECT YEAR(order_date) yr, MONTH(order_date) mo, SUM(sales_amount) total_sales  
    FROM gold.fact_sales WHERE order_date IS NOT NULL GROUP BY YEAR(order_date), MONTH(order_date)  
)  
SELECT yr, mo, total_sales, total_sales - LAG(total_sales) OVER (ORDER BY yr, mo) AS mom_change  
FROM monthly;
```

- Ordering by yr, mo together (not a single date) keeps the LAG() window correctly chronological across year boundaries.

#177 Revenue concentration (Pareto check)

ADVANCED

Compute each product's cumulative share of total revenue, ordered by revenue descending, to test an 80/20 concentration pattern.

SOLUTION

```
WITH prod_rev AS (  
    SELECT p.product_name, SUM(f.sales_amount) AS revenue  
    FROM gold.fact_sales f JOIN gold.dim_products p ON f.product_key = p.product_key  
    GROUP BY p.product_name  
)  
SELECT product_name, revenue,  
    SUM(revenue) OVER (ORDER BY revenue DESC ROWS UNBOUNDED PRECEDING) * 100.0 / SUM(revenue) OVER () AS cumulative_pct  
FROM prod_rev;
```

- A cumulative-percentage window function is the standard way to test whether a small share of products drives most revenue.

#178 New vs. returning customers by month

ADVANCED

For each month, classify customers as 'new' (first order that month) or 'returning', and count each group.

SOLUTION

```
WITH first_order AS (  
    SELECT customer_key, MIN(order_date) AS first_order_date FROM gold.fact_sales GROUP BY customer_key  
)  
SELECT YEAR(f.order_date) yr, MONTH(f.order_date) mo,  
    SUM(CASE WHEN f.order_date = fo.first_order_date THEN 1 ELSE 0 END) AS new_customers,  
    COUNT(DISTINCT CASE WHEN f.order_date <> fo.first_order_date THEN f.customer_key END) AS returning_customers  
FROM gold.fact_sales f JOIN first_order fo ON f.customer_key = fo.customer_key  
GROUP BY YEAR(f.order_date), MONTH(f.order_date) ORDER BY yr, mo;
```

- Comparing each order's date to the customer's own first-order date is the key trick behind new-vs-returning segmentation.

#179 Average shipping lead time overall

BEGINNER

Calculate the overall average number of days between order_date and shipping_date.

SOLUTION

```
SELECT AVG(DATEDIFF(DAY, order_date, shipping_date)) AS avg_lead_time_days FROM gold.fact_sales WHERE order_date IS NOT NULL AND shipping_date IS NOT NULL;
```

- A single average lead-time KPI is a common line item on an operations dashboard.

#180 Sales by gender

BEGINNER

Total sales_amount broken out by customer gender.

SOLUTION

```
SELECT c.gender, SUM(f.sales_amount) AS total_sales
FROM gold.fact_sales f JOIN gold.dim_customers c ON f.customer_key = c.customer_key
GROUP BY c.gender;
```

- Demographic breakdowns like this are simple once the dimension attribute is already clean.

#181 Sales by marital status

BEGINNER

Total sales_amount broken out by customer marital status.

SOLUTION

```
SELECT c.marital_status, SUM(f.sales_amount) AS total_sales
FROM gold.fact_sales f JOIN gold.dim_customers c ON f.customer_key = c.customer_key
GROUP BY c.marital_status;
```

- The normalized 'Single'/'Married'/'n/a' labels from the silver layer make this grouping directly readable.

#182 Category share of total revenue

INTERMEDIATE

Calculate each category's percentage contribution to total revenue using a window function.

SOLUTION

```
SELECT p.category, SUM(f.sales_amount) AS revenue,  
       SUM(f.sales_amount) * 100.0 / SUM(SUM(f.sales_amount)) OVER () AS pct_of_total  
FROM gold.fact_sales f JOIN gold.dim_products p ON f.product_key = p.product_key  
GROUP BY p.category;
```

- SUM(...) OVER () with no PARTITION BY computes the grand total alongside each group's subtotal in one pass.

#183 Customers with declining year-over-year spend

ADVANCED

Find customers whose total spend decreased from one year to the next.

SOLUTION

```
WITH yearly AS (  
  SELECT customer_key, YEAR(order_date) yr, SUM(sales_amount) total_sales  
  FROM gold.fact_sales WHERE order_date IS NOT NULL GROUP BY customer_key, YEAR(order_date)  
)  
SELECT customer_key, yr, total_sales, LAG(total_sales) OVER (PARTITION BY customer_key ORDER BY yr) AS prev_year  
FROM yearly  
-- filter in an outer query: WHERE total_sales < prev_year
```

- Partitioning the LAG() by customer_key keeps each customer's year-over-year comparison independent of the others.

#184 Products with zero sales

BEGINNER

Find products in gold.dim_products that have never appeared in gold.fact_sales.

SOLUTION

```
SELECT p.product_name FROM gold.dim_products p  
LEFT JOIN gold.fact_sales f ON p.product_key = f.product_key  
WHERE f.product_key IS NULL;
```

- A dimension-to-fact anti-join is the standard way to find 'never sold' items for a catalog review.

#185 One-row KPI summary

INTERMEDIATE

Build a single-row KPI summary: total revenue, total orders, total customers, and average order value.

SOLUTION

```
SELECT
  SUM(sales_amount) AS total_revenue,
  COUNT(DISTINCT order_number) AS total_orders,
  COUNT(DISTINCT customer_key) AS total_customers,
  SUM(sales_amount) * 1.0 / COUNT(DISTINCT order_number) AS avg_order_value
FROM gold.fact_sales;
```

- A one-row KPI query like this is often the exact result set behind a dashboard's top summary tiles.

I. ETL Orchestration & Auditing

Tasks 186-200 • 15 practice tasks

#186 Load procedure skeleton with timing

INTERMEDIATE

Write a stored procedure skeleton with TRY/CATCH and PRINT statements timing the start and end of a load step.

SOLUTION

```
CREATE PROCEDURE silver.load_silver AS
BEGIN
    BEGIN TRY
        DECLARE @start_time DATETIME = GETDATE();
        -- load steps here
        PRINT 'Load Duration: ' + CAST(DATEDIFF(SECOND, @start_time, GETDATE()) AS NVARCHAR) + ' seconds';
    END TRY
    BEGIN CATCH
        PRINT 'ERROR: ' + ERROR_MESSAGE();
    END CATCH
END;
```

- This TRY/CATCH-with-timing skeleton is reused around every table load in the production bronze and silver procedures.

#187 Row-count logging per table

INTERMEDIATE

Extend a load step to PRINT the number of rows inserted into silver.crm_cust_info.

SOLUTION

```
INSERT INTO silver.crm_cust_info (cst_id, cst_key)
SELECT cst_id, cst_key FROM bronze.crm_cust_info WHERE cst_id IS NOT NULL;
PRINT '>> Rows inserted: ' + CAST(@@ROWCOUNT AS NVARCHAR);
```

- @@ROWCOUNT immediately after an INSERT captures exactly how many rows that statement affected.

#188 Design an audit log table

INTERMEDIATE

Design a schema for an etl_load_log table that tracks batch runs: layer, table_name, start_time, end_time, row_count, status.

SOLUTION

```
CREATE TABLE dbo.etl_load_log (  
    log_id INT IDENTITY PRIMARY KEY,  
    layer NVARCHAR(20), table_name NVARCHAR(100),  
    start_time DATETIME, end_time DATETIME,  
    row_count INT, status NVARCHAR(20), error_message NVARCHAR(4000)  
);
```

- A structured audit log turns scattered PRINT statements into a queryable operational history.

#189 Write audit rows instead of only printing

ADVANCED

Modify a load step to INSERT a row into etl_load_log instead of (or in addition to) PRINTing progress.

SOLUTION

```
DECLARE @start DATETIME = GETDATE();  
TRUNCATE TABLE silver.crm_cust_info;  
INSERT INTO silver.crm_cust_info (cst_id, cst_key) SELECT cst_id, cst_key FROM bronze.crm_cust_info;  
INSERT INTO dbo.etl_load_log (layer, table_name, start_time, end_time, row_count, status)  
VALUES ('silver', 'crm_cust_info', @start, GETDATE(), @@ROWCOUNT, 'SUCCESS');
```

- Persisting the log means it survives after the session ends — PRINT output alone is only visible during execution.

#190 Log errors in the CATCH block

ADVANCED

Extend the CATCH block to write a FAILED row into etl_load_log with the captured error message.

SOLUTION

```
BEGIN CATCH  
    INSERT INTO dbo.etl_load_log (layer, table_name, start_time, end_time, status, error_message)  
    VALUES ('silver', 'crm_cust_info', @start, GETDATE(), 'FAILED', ERROR_MESSAGE());  
END CATCH;
```

- Logging failures with the same structure as successes makes it trivial to query 'show me every failed load this month'.

#191 Chain bronze then silver loads

INTERMEDIATE

Write a wrapper procedure that calls `bronze.load_bronze` followed by `silver.load_silver` in sequence.

SOLUTION

```
CREATE PROCEDURE dbo.run_full_load AS
BEGIN
    EXEC bronze.load_bronze;
    EXEC silver.load_silver;
END;
```

- A single orchestrating procedure is the simplest form of pipeline scheduling before adopting a dedicated orchestrator.

#192 Abort silver load if bronze is empty

ADVANCED

Add a guard clause at the start of the silver load that raises an error and stops if any required bronze table is empty.

SOLUTION

```
IF NOT EXISTS (SELECT 1 FROM bronze.crm_cust_info)
BEGIN
    RAISERROR('Bronze layer is empty - aborting silver load.', 16, 1);
    RETURN;
END;
```

- Failing fast on an empty upstream table prevents a silver load from silently truncating good data with nothing to replace it.

#193 Incremental MERGE-based load

ADVANCED

Convert the full-refresh `silver.crm_cust_info` load into an incremental load using `MERGE`, matched on `cst_id`.

SOLUTION

```
MERGE silver.crm_cust_info AS tgt
USING (SELECT cst_id, cst_key, cst_firstname FROM bronze.crm_cust_info WHERE cst_id IS NOT NULL) AS src
ON tgt.cst_id = src.cst_id
WHEN MATCHED THEN UPDATE SET cst_key = src.cst_key, cst_firstname = src.cst_firstname
WHEN NOT MATCHED THEN INSERT (cst_id, cst_key, cst_firstname) VALUES (src.cst_id, src.cst_key, src.cst_firstname);
```

- `MERGE` lets a load update existing rows and insert new ones in a single statement, replacing the `TRUNCATE+INSERT` full-refresh pattern.

#194 Watermark table for incremental bronze loads

ADVANCED

Design a simple watermark table tracking the last successfully loaded file/date per bronze table, to support incremental extraction.

SOLUTION

```
CREATE TABLE dbo.etl_watermark (  
    table_name NVARCHAR(100) PRIMARY KEY,  
    last_loaded_date DATETIME  
);  
-- Before loading: SELECT last_loaded_date FROM dbo.etl_watermark WHERE table_name = 'crm_sales_details';  
-- After loading: UPDATE dbo.etl_watermark SET last_loaded_date = GETDATE() WHERE table_name =  
'crm_sales_details';
```

- A watermark table is the standard low-tech mechanism for 'only load what's new since last time' incremental pipelines.

#195 Query recent load durations

BEGINNER

Using the etl_load_log table, find the duration in seconds of the last 10 load runs.

SOLUTION

```
SELECT TOP (10) table_name, DATEDIFF(SECOND, start_time, end_time) AS duration_seconds FROM dbo.etl_load_log ORDER BY start_time DESC;
```

- Trending load duration over time is an easy way to catch a pipeline that's gradually slowing down.

#196 Data quality gate before promoting to gold

ADVANCED

Design a gate query that returns a single PASS/FAIL flag, failing the batch if any silver DQ check (from Category C) returns failing rows.

SOLUTION

```
DECLARE @fail_count INT;  
SELECT @fail_count = COUNT(*) FROM (  
    SELECT cst_id FROM silver.crm_cust_info GROUP BY cst_id HAVING COUNT(*) > 1  
) x;  
IF @fail_count > 0  
    RAISERROR('Data quality gate failed - duplicate cst_id found.', 16, 1);
```

- A DQ gate that halts the pipeline is far safer than promoting known-bad data into the gold layer and finding out later.

#197 Transaction-safe silver load

ADVANCED

Wrap the silver.crm_cust_info TRUNCATE + INSERT in an explicit transaction with rollback on error.

SOLUTION

```
BEGIN TRY
  BEGIN TRANSACTION;
  TRUNCATE TABLE silver.crm_cust_info;
  INSERT INTO silver.crm_cust_info (cst_id, cst_key) SELECT cst_id, cst_key FROM bronze.crm_cust_info;
  COMMIT TRANSACTION;
END TRY
BEGIN CATCH
  ROLLBACK TRANSACTION;
  PRINT 'Load failed and was rolled back: ' + ERROR_MESSAGE();
END CATCH;
```

- Wrapping TRUNCATE+INSERT in a transaction means a mid-load failure leaves the table exactly as it was before the load started.

#198 Trace one order through the whole pipeline

INTERMEDIATE

Prove the bronze -> silver -> gold lineage for a single order_number by finding it in every relevant layer.

SOLUTION

```
SELECT * FROM bronze.crm_sales_details WHERE sls_ord_num = 'S043697';
SELECT * FROM silver.crm_sales_details WHERE sls_ord_num = 'S043697';
SELECT * FROM gold.fact_sales WHERE order_number = 'S043697';
```

- Tracing a single record end-to-end is one of the fastest ways to debug 'why does this number look wrong in the report'.

#199 Parameterize load_bronze by source

INTERMEDIATE

Add a parameter to bronze.load_bronze so it can optionally load only CRM tables, only ERP tables, or both.

SOLUTION

```
CREATE PROCEDURE bronze.load_bronze @source NVARCHAR(10) = 'ALL' AS
BEGIN
  IF @source IN ('ALL', 'CRM') BEGIN /* load CRM tables */ END
  IF @source IN ('ALL', 'ERP') BEGIN /* load ERP tables */ END
END;
```

- A source filter parameter is handy when one source system's file arrives late and you don't want to block the other.

#200 Check silver-layer freshness

BEGINNER

Write a query returning the most recent dwh_create_date per silver table, to confirm the last load actually ran.

SOLUTION

```
SELECT 'crm_cust_info' AS table_name, MAX(dwh_create_date) AS last_loaded FROM silver.crm_cust_info
UNION ALL SELECT 'crm_prd_info', MAX(dwh_create_date) FROM silver.crm_prd_info
UNION ALL SELECT 'crm_sales_details', MAX(dwh_create_date) FROM silver.crm_sales_details;
```

- A freshness check like this is exactly what an automated monitoring alert would run every morning before the business day starts.